Are We Really Making Much Progress in Text Classification? A Comparative Review

Ansgar Scherp*, Andor Diera, Fabian Karl, Bao Xin Lin†, Bhakti Khera,
Tim Meuser, Tushar Singhal, and Lukas Galke Poech*

***Abstract*—We survey the literature on single-label, multi-label, and hierarchical text classification and provide a quantitative comparison of methods categorized into bag-of-words, sequence-based, and graph- or hierarchy-based approaches. Despite a recent surge in graph-based methods, they do not provide an improvement over fine-tuned transformer models on most evaluated datasets. Decoder-only generative language models show promise in few-shot in-context learning, but appear to lag behind fine-tuned language models when sufficient training data is available. The amount of training data needed for a fine-tuned language model to exceed the performance of a generative model is task-dependent. We further highlight the variance in reported numbers across the literature when applying the same model to the same dataset, which can be traced to the use of different hyperparameter values, such as the fine-tuning learning rate. For practitioners, we recommend using a fine-tuned language model when sufficient training data is available. Otherwise, a frozen generative model, enhanced by few-shot in-context learning or reasoning, is preferable.**
Data availability: The source code and further information are available at https://github.com/ascherp/text-classification-survey.

***Index Terms*—Data mining, text mining, text classification, neural networks, machine learning**

I. Introduction

A. Motivation and Background

Text classification is the task of assigning a categorical label or multiple of such labels to a given text unit [1], [2]. It is a central task in natural language processing (NLP), with numerous practical applications, such as classifying scholarly documents, social media posts, news articles, or email spam. Unsurprisingly, to this day, text classification remains a very active research field, with new methods appearing every week, as reflected by recent surveys [3]–[11].

Besides the rapid pace of research in text classification, conceptually new approaches have also emerged that are not yet sufficiently covered in existing surveys. Those include the use of graph neural networks (GNNs) to process text, which attracts increasing attention from researchers, and increasingly prevalent large language models (LLMs), which are often assumed to be the state-of-the-art in all natural language processing tasks. Although there are surveys with good coverage of new methods, e. g., focusing on LLMs [3] or GNNs [12], those then lack a quantitative comparison. Other surveys perform a quantitative comparison but focus on single-label classification and use their own splits instead of the established splits per dataset [13]. This survey covers both of these new families alongside classical approaches and provides a quantitative comparison across multi-class (or single-label), multi-label, and hierarchical text classification.

Many new methods for text classification are based on graph neural networks (GNNs) [14]. Common to these GNN-based approaches is that they first generate a synthetic graph from the corpus that contains text-augmented vertices and edges with information on word and document co-occurrences, e. g., [15], [16]. Second, the GNN is trained on this graph to carry out the text classification task. In hierarchical text classification, where classes are organized along a thematic hierarchical thesaurus [1], GNNs are often used to encode the label hierarchy, e. g., [17], [18].

Another group of text classification methods is based on language models, which can be organized in encoder-only, encoder-decoder, and decoder-only models [19]. Encoder-only pre-trained language models such as BERT [20] took big strides in many natural language processing tasks, including categorical text classification [21]. The encoder-only transformer language models were followed by encoder-decoder variants like T5 [22] and decoder-only generative LLMs such as the GPT models [23], [24]. Decoder-only transformer language models focus on text generation with remarkable in-context learning abilities. This makes them strong zero-shot and few-shot models [24]–[27]. Yet, whether in-context learning LLMs are generally superior to fine-tuned small language models remains an open question [28]–[34].

Decoder-only language models are usually much larger than masked language models [24], which is because the causal language model objective architecture (left-to-right prediction) facilitates scalability through diagonal masking of the attention matrices – enabling the model to get an error signal for each token in a batch. Thus, we increasingly find the distinction between SLMs (small language models) and LLMs, e. g., [35]–[37], which is based on an arbitrarily chosen parameter count.

We argue that a key distinction between SLMs and LLMs, particularly for classification tasks, lies in whether they are amenable to fine-tuning. The encoder-only and encoder-decoder SLMs (e. g., BERT and T5) are based on masked language modeling (or masked span prediction for T5 respectively) and typically undergo subsequent task-specific fine-tuning. Decoder-only LLMs (e. g., GPTs) are pre-trained using a causal language modeling objective. Sufficiently powerful decoder-only models are capable of tackling new tasks based on instructions in their context window at inference time [24]. That is, while still possible, fine-tuning is not strictly needed

L. Galke Poech is with the University of Southern Denmark. A. Scherp, A. Diera, F. Karl, B. Lin, B. Khera, T. Meuser, and T. Singhal are with the University of Ulm, Germany. *L. Galke Poech and A. Scherp contributed equally. †Deceased.

for decoder-only models to facilitate downstream tasks like text classification. Task-specific fine-tuning with LLMs is typically more prohibitive there due to the higher number of parameters. Thus, most current text classification approaches using LLMs rely on few-shot in-context learning and dedicated prompting schemes [25].

B. What is the Problem? Why is our Study Needed?

Encoder-only language models like BERT have led to major improvements to the state of the art on many NLP tasks, including categorical text classification [21]. We ask whether the numerous newly proposed methods provide substantial improvements over encoder-only language models. We pinpoint this question to three aspects analyzed in this survey:

- *Use of Graphs:* The use of GNNs has been specialized for text classification in two ways: First, using synthetic graphs induced from the text corpus, e. g., TextGCN [15]. Those synthetic graphs may not provide additional information beyond what a neural network can directly extract from the text [21]. Second, graphs are provided as additional input for hierarchical classification and processed by explicit graph-encoders, e. g., HiAGM [38]. Graph encoders may not provide added benefits to the task of hierarchical classification [39], [40].
- *Large Language Models:* The literature suggests that generative LLMs do not generally improve over encoder-only SLMs for text classification tasks [25], [28], [41]–[44]. This comes despite the advantages of generative LLMs of high capacity, prompt engineering techniques [25], [45], instruction fine-tuning [27], and prompt tuning [46], [47].
- *Comparability of Methods:* A general challenge when assessing the performance of methods is the comparability of the results. Besides properly reporting the used datasets, splits, preprocessing, etc. an important question is if baselines are properly optimized [48], [49]. It is especially challenging to properly compare methods for text classification when so many new papers appear using GNNs and language models.

C. Methodology

We extensively review literature on modern and classical machine learning methods for single-label, multi-label, and hierarchical text classification. Based on the literature search, we derive the families of methods for text classification.

- Methods based on Bag of Words (BoW) using, e. g., a support vector machine or a multi-layer perceptron [21].
- Methods considering text as a sequence of tokens, e. g., the encoder-only BERT [20], encoder-decoder T5 [22], or decoder-only GPT [24],
- Graph-based methods that employ graph neural networks on synthetic graphs like TextGCN [15] and hierarchy-based text classification methods like HGCLR [18].

We identify established single-label and multi-label benchmark datasets for our comparison and identify the top-performing methods. We carefully assess the validity of the results for each method and each paper in the literature. We check, among others, the train-test split used (whether they deviate from established benchmark splits), number of classes considered, hyperparameter values (are they provided and comparable), whether the baselines are optimized, metrics applied, and if there is any unusual preprocessing of the datasets that may have influenced the results.

We aggregate results from the literature, complement them with experiments where relevant, and provide a systematic comparison of methods across different method families.

D. Key Results

Our comparative survey shows that fine-tuned transformer models are generally strong models for text classification. Despite their sheer number of parameters and training, LLMs based on in-context learning generally do not outperform fine-tuned SLMs in text classification. Even when pushing the limit of using such generative language models by applying tricks such as advanced prompting techniques [25] or using ensembles [41], the performance does not, or only marginally on individual datasets, outperform those of encoder-only SLMs. For example, it requires an ensemble of fine-tuned Llama models to reach or slightly outperform an encoder-only SLM on two benchmark datasets [41].

We attribute these observations to two main factors. First, task-specific fine-tuning is still an important distinctive criterion. The encoder-only models like BERT are pre-trained with the masked language modeling objective and fine-tuned with a classifier head for specific datasets on the text classification task. Second, the left-to-right attention mask seems to be suboptimal for text classification tasks. This again relates well to the distinction between discriminative and generative approaches [50] – the former focusing on the decision boundary directly, i. e., $p(y|x)$, which we interpret here as task-specific fine-tuning, and the latter aiming to learn the full joint distribution $p(x, y)$, which we interpret here as in-context learning.

For the GNN-based methods, despite the huge number of methods developed in recent years, the idea of exploiting a synthetically induced graph to improve text classification has not lived up to its promise. Many graph-based methods such as [15], [51], [52] are not only outperformed by simply applying BERT but already fall below simple baselines such as a logistic regression or a simple classifier such as a multi-layer perception on a bag-of-words representation [21], [52]. Such baseline models already extract relevant information for text classification from the raw text [1]. Adding a synthetically generated graph from the corpus generally does not provide additional information that a neural network can exploit [21].

A worrying observation is the lack of strong baselines and/or not properly tuning them appropriately. Like Dacrema, Cremonesi, and Jannach [48] observed for neural recommender systems that "works can be outperformed at least on some datasets by conceptually and computationally simpler algorithms", the reason is also that baselines are not properly optimized [53] ("everyone's a winner"). Classical machine learning methods such as support vector machines, logistic regression, and multi-layer perceptrons are largely ignored in

papers published in the last years, despite showing consistently strong results across various datasets [21], [52], [54].

There are cases where even the baseline's most basic hyperparameter, the learning rate (or fine-tuning learning rate, in the case of BERT models), is not properly considered. We show this in the example of BERT and its variants. Despite running the same BERT model on the same datasets and splits, the numbers from the literature show a discrepancy in BERT's performance up to a 13 points in accuracy. Generally, such discrepancies may be attributed to the learning rate, preprocessing, batch size, implementation details, random seeds, and tuning efforts. We analyze the effect of the learning rate in more detail.

Following studies in other areas of machine learning, we conclude that strong baselines must be used and their hyperparameters properly selected as it is an important means to argue about *true* scientific advancement [48], [55], and also their usefulness in practical settings.

E. Remainder

Below, we introduce our survey methodology. The experimental apparatus, including the considered datasets, and the procedure for running our experiments are described in Section VI. Sections III, IV, and V provide an overview of the literature in bag-of-words approaches, sequence-based approaches, and graph-based approaches, respectively. The results of our quantitative comparison are presented in Section VII. Subsequently, we discuss the findings in Section VIII, limitations in Section IX, and provide future directions in Section X, before we conclude.

II. Methodology

This survey covers single-label text classification, multi-label text classification, and hierarchical text classification – covering published methods up to May 2026. We do not consider *extreme* multi-label classification, where the focus is on dealing with a very large label space, which requires a different set of specialized techniques [56]–[59] and different metrics (ranking metrics rather than classification metrics). We leave this comparison for future work.

In a first step, we have collected and analyzed recent surveys on single-label, multi-label, and hierarchical text classification and searched for research papers that include comparison studies [6]–[11], [21], [60]–[69]. These cover the range from classical machine learning models to deep classification models. Second, we have screened for literature in key NLP and artificial intelligence venues.

Based on this input, we have determined the families of methods and benchmark datasets used (see Tables I and II). The categorization of the methods is mainly based on the method signature, distinguishing whether methods operate on BoW, sequence, or graph-structured input [21]. We extend the categorization by further distinguishing sequence-based methods into classical recurrent and convolution models, encoder-only (language) models, encoder-decoder models, and decoder-only models. Furthermore, we distinguish between two types of graph-based methods: First, approaches that employ a graph structure derived from a corpus of text documents, which we call synthetic text-graph approaches. Second, hierarchy-based methods that use a graph model to encode the hierarchical structure among classes.

We focus our analysis on methods that show strong performance and include them in our study. Comparing results from different studies comes with limitations regarding comparability of results. To ensure comparability to the extent possible, we have verified that the same train-test split of the datasets is used for all methods and that the same metrics are used. We have also checked whether the datasets have been modified (e. g., use only a subset of the classes or have removed samples) to avoid bias and mistakenly give advantages.

We do not consider papers that do not allow for a fair comparison with the state of the art. Reasons include that they

- used different benchmark datasets only like [3], [42]–[44], [70]–[100],
- modified the datasets to use a different number of classes as done in [101]–[103],
- employed different train-test splits like [13], [33], [64], [104]–[110],
- conduct their own sampling of the train-test splits instead of using standardized splits [111],
- perform data augmentation from external knowledge bases [87],
- train-test splits are not reported [109], [112], [113],
- used fewer training examples [61], [87], [107], [114]–[118], or
- used different evaluation measures [119], [120].

The rationales for why certain changes are made were not always clear in the literature. However, it reflects a general problem of comparability in machine learning research [49].

III. BoW-based Methods

Under pure BoW-based (Bag of Words) text classification, we denote approaches that operate only on a multiset of words (or tokens) from the input document. Given paired training examples $(\boldsymbol{x}, y) \in \mathcal{D}$, each consisting of a vector that holds the frequency of words in its components $\boldsymbol{x} \in \mathbb{R}^{n_{\text{vocab}}}$, which is commonly referred to as a bag (multiset) of words, and a class label $y \in \mathbb{Y}$. The goal is to learn a generalizable function $\hat{\boldsymbol{y}} = f_\theta^{(\text{BoW})}(\boldsymbol{x})$ with parameters θ such that $\arg\max(\hat{\boldsymbol{y}})$ is the true label y for input $\boldsymbol{x}$. For multi-label classification, the BoW-based model considers multiple class labels. Instead of using $\arg\max(\hat{\boldsymbol{y}})$ to decide on a label, a binary sigmoid output is commonly used per label, along with a threshold λ that determines whether the corresponding class will be assigned. The multi-label model is trained with a binary cross-entropy loss instead of a categorical cross-entropy. As output, the multi-label classifier can produce between 0 and $|\mathbb{Y}|$ many labels [1], i. e., it is possible that no label is predicted.

A. Classical BoW Methods

Classical machine learning methods that operate on a BoW-based input are extensively discussed in surveys and comparison studies [10], [11], [52], [68]. These studies show that the best-performing classical models are Support Vector

Machines (SVM) and logistic regression (LR). Especially, the strong performance of logistic regression is astonishing. For instance, Ragesh et al. [52] have shown that logistic regression outperforms the advanced graph-based TextGCN [15] method. The CFE-IterativeAdditive method uses a linear SVM with term-based substring matching (from the documents) to the class names [121]. It uses this label matching of the terms to adapt the global IDF weights iteratively, denoted as TF-ICF.

B. Deep BoW Methods

With more advanced BoW methods, Galke et al. [21] have found that an MLP on a bag-of-words representation of the text outperforms many graph-based approaches. Earlier approaches are mainly based on pre-trained word embeddings [122], [123]. For instance, Iyyer et al. [124] proposed Deep Averaging Networks (DAN), a combination of word embeddings and deep feedforward networks. DAN is an MLP with one to six hidden layers, non-linear activation, dropout, and AdaGrad as an optimization method. The results suggest that pre-trained embeddings such as GloVe [123] would be preferable over randomly initialized neural bag-of-words [125]. In fastText [126], [127], a linear layer is used on top of pre-trained embeddings for classification. Furthermore, Shen et al. [55] explore different pooling variants for the input word embeddings and find that their Simple Word Embedding Models (SWEM) can rival approaches based on recurrent (RNN) and convolutional neural networks (CNN). Note that approaches such as fastText and SWEM that apply a logistic regression on top of pre-trained word embeddings share a similar architecture as an MLP with one hidden layer. However, the standard training protocol involves pre-training the word embedding on large amounts of unlabeled text and then freezing the word embeddings while training the logistic regression [122].

IV. Sequence-based Methods

As sequence-based methods, we consider recurrent and convolutional neural networks, SLMs (e. g., BERT and T5) and LLMs (e. g., GPT-3.5). These approaches aim for *contextualized* word representations, for which nearby words and word order are taken into account. The model signature for sequence-based methods is $\hat{y} = f_\theta^{(\text{sequence})}(\langle x_1, x_2, \ldots, x_k \rangle)$, where k is the (maximum) sequence length.

A. Recurrent and Convolutional Neural Networks

Before the era of pre-trained language models, recurrent neural networks (RNNs) were a natural choice for any NLP task. In particular, LSTMs [128] enhanced with an attention mechanism [129] and bidirectional processing [130], were considered the state of the art for NLP tasks before transformers. For instance, SGM [131] is a model for multi-label classification based on a bidirectional LSTM [128] with an attention mechanism, which has later been extended to use T5 as a backbone language model [132]. Word embeddings, such as GloVe [123] have been used with CNNs [133] as well as Bi-LSTM [134]. Lyu and Liu [135] combine a 2D-CNN with a bidirectional RNN.

B. Encoder-only Small Language Models

The transformer [136], originally developed for machine translation, introduced a key-value self-attention mechanism and had an immense impact on language modeling. Transformer-based language models are pre-trained with a self-supervised training objective on large text corpora and can be subsequently fine-tuned with a supervised objective on a specific task. Generally, transformer-based language models can be categorized into encoder-only models, encoder-decoder models, and decoder-only models [19]. BERT and its follow-up approaches are encoder-only language models and are trained with the masked-language modeling objective [20]. Their main purpose is encoding the text and performing a classification or regression task with a dedicated module (e. g., a linear output layer) on top of the encoder. The output layer is trained from scratch (i. e., from random initialization) during fine-tuning.

Popular follow-up works of BERT are RoBERTa [137], DistilBERT [138], ALBERT [139], DeBERTa [140], and ERNIE 2.0 [141] *inter alia*. RoBERTa improves the pre-training procedure of BERT and removes the next sentence prediction objective. DeBERTa improves upon BERT through a disentangled attention mechanism and corresponding masking techniques. DeBERTaV3 [142] is a modified DeBERTa with an ELECTRA-style pre-training [143]. DistilBERT [138] is a distilled version of BERT with 40% reduced parameters and 60% faster inference time that retains 97% of BERT's performance on the GLUE benchmark [144]. ALBERT reduces the memory footprint and increases the training speed of BERT for improved scalability. DeBERTa introduces a disentangled attention mechanism, i. e., keeping word position and content embeddings separate, and an enhanced mask decoder, which introduces absolute position encodings to the final softmax. ERNIE 2.0 employs a continual multi-task learning strategy. Whenever a new task is introduced, the previous model parameters are used for initialization, and the new task is added to the multi-task learning objective. The AM-BERT and AM-RoBERTa models augment transformer fine-tuning for text classification with two attention matrices based on keyphrases and on the input's dependency graph [145]. Finally, ModernBERT is a refurbished variant of the BERT model integrating various optimizations developed over the years and providing an input length of $8,192$ tokens [146].

Pre-trained encoder-only language models have also found their way into hierarchical text classification. For instance, HBGL [17] uses BERT to represent the text and represent the hierarchically organized classes. For the classes, HBGL first learns the label embeddings from the global hierarchy, i. e., the taxonomy. Specifically, the label embeddings are learned by employing a masked language modeling objective on sequences that correspond to paths in the label hierarchy. Subsequently, it learns to predict the document labels one by one in the order of the local hierarchy, i. e., level-wise from the root to the most specific label in the taxonomy. By this, HBGL exploits the hierarchy of the labels as defined in the taxonomy and treats the label generation as a multi-label text classification task in a similar way as Nam et al. [147]

did with sequence-to-sequence models trained from scratch. Thus, HBGL is essentially a *hierarchy-aware* sequence-based transformer model. However, as described above, the key ingredient of HBGL is making use of the sequence-based model BERT. HBGL *does not use an external graph encoder* for representing the taxonomy. In contrast, the graph-based methods (described in Section V) always use an *explicit* graph encoder for representing the taxonomy, most commonly a graph neural network, independent of what other model is being used (e. g., a CNN, BERT, or other). Since HBGL's core is heavily based on BERT and does not have an explicit graph encoder, we consider it a sequence-based transformer method.

C. Encoder-decoder Small Language Models

Examples of encoder-decoder language models include BART [148] and T5 [22], which combine ideas from masked language models and generative transformers. The idea is to cast downstream tasks, including classification, into a text-to-text framework with task-specific prompt prefixes. Although these models generate text, they can also be used for classification tasks by interpreting generated tokens as class labels (generative variant, as in [40]) or through fine-tuning a dedicated new classifier head (discriminative variant). In multi-label classification, text-to-text (= sequence-to-sequence) models have been used even before the era of large language models [147] to facilitate the prediction of multiple labels.

The encoder-decoder model Seq2Tree adopts T5 to perform hierarchical text classification [132]. It uses a depth-first search approach to linearize the label hierarchy in order to encode it as a sequence in the model. RADAr is a sequence-to-sequence model that uses RoBERTa as a text encoder and a custom two-layer Transformer-based decoder for hierarchical text classification [40]. In contrast to hierarchical text classification methods, see Section V-B, RADAr does not operate on a given dataset taxonomy.

D. Large Language Models

Decoder-only language models such as GPT-3 [24] and Llama3 [149] are trained with a left-to-right, i. e., causal language modeling, objective. This makes decoder-only models most suitable for text generation. However, decoder-only models are very flexible and can also be used to carry out other downstream tasks, including text classification [24], [150]. This is done by specifying the task in natural language and providing a few examples in the prompt [24]. This practice, known as in-context learning, does not require updating the model. In-context learning has led to increased interest in the design of and working with prompts, such as Chain-of-Thought (CoT) prompting [45] *inter alia*.

Sun et al. [25] evaluated GPT-3.5+CoT on single-label text classification. They also introduced the prompting strategy Clue and Reasoning Prompting (CARP), where the instructions consist of first finding relevant clues in the text input and then providing a classification result based on the clues along with an explanation. The authors evaluate GPT-3.5, GPT-3.5+CoT, and GPT-3.5+CARP with two different samplers for selecting the in-context examples. The samplers are a uniform sampler and a RoBERTa model fine-tuned on a generic natural language inference task. Using RoBERTa representations of the training documents, the sampler employs a kNN search on the examples to sample more representative documents per class to be included in the prompt. The best-performing CARP variant employs a 16-shot RoBERTa sampler with a majority vote over multiple runs of prompting GPT-3.5, which we denote as GPT-3.5+CARP+vote.

Prompt boosting has shown promising results for single-label classification [151], where differently prompted LLMs form an ensemble with an adaptive boosting algorithm. However, the few-shot performance of large language models without any fine-tuning is still lower than that of fine-tuned small language models [33]. Beyond prompting techniques, Zhang et al. [41] have experimented with fine-tuning an ensemble of Llama-2 [152] models for text classification, leading to competitive scores. Li et al. [34] fine-tune a Llama-2 model [152] without causal masking such that both left and right context can be considered during self-attention.

In the end, it is currently unknown whether in-context learning with a large language model is sufficient for a given task or whether fine-tuning is needed.

V. Graph-based Methods

Graphs can serve several purposes in text classification: One is to consider the input data as a graph (i. e., the documents, their words), which we call synthetic text-graph approaches to distinguish them from graphs in which the structure has a natural interpretation (e. g., citations graphs). Another purpose is considering an additional label hierarchy as input to the model [1] (i. e., classes organized in a hierarchy), which we call hierarchy-based methods.

A. Synthetic Text-Graph Methods

Using graphs induced from text has a long history in text classification. An early work is the term co-occurrence graph of the KeyGraph algorithm [153]. The graph is split into segments, representing the key concepts in the document. Co-occurrence graphs have also been used for automatic keyword extraction [154] and classification [155]. Modern methods exploit this idea of a graph induced from the text. The text corpus is first transformed into a graph, which is then fed as input into a graph neural network (GNN) [14].

The synthetic text-graph approaches to text classification first set up a *synthetic* graph based on the text corpus $\mathcal{D}$ such that an adjacency matrix is created from a document corpus $\hat{\boldsymbol{A}} := \text{make-graph}(\mathcal{D})$. The graph is composed of word nodes and document nodes, each receiving its own embedding (by setting $\boldsymbol{X} = \boldsymbol{I}$). For example, in TextGCN, the graph is created from word-word edges (modeled by pointwise mutual information) and word-document edges (based on TF-IDF scores). Then, a parameterized function $f_\theta^{(\text{graph})}(\boldsymbol{X}, \hat{\boldsymbol{A}})$ is learned that uses the graph as input, where $\boldsymbol{X}$ are the node features. In the case of TextGCN, the function is a standard graph convolutional network [156]. Note that graph-based approaches such as TextGCN disregard word order, similar to the BoW-based models described above.

Among others, methods that follow this synthetic text-graph approach include TextGCN [15], TensorGCN [51], HeteGCN [52], HyperGAT [157], HGAT [158], DADGNN [159], STGCN [160], SHINE [118], and AGGNN [161]. While many of these methods are strictly transductive such as HeteGCN and TensorGCN, other methods are inductive [16], [133], [162], [163] or can be adapted to become inductive [52]. Transductive models need access to the unlabeled test documents at training time. This requires computing the graph also on the documents of the test set and making this information available during training (but without the labels from the test set). In contrast, inductive models can be applied to new data. Here, the graph induced from the text is computed only on the training set.

Transductive training has inherent drawbacks as the models cannot be applied to new documents. For example, in TextGCN's original transductive formulation, the entire graph, including the unlabeled test set, must be available for training. This may be prohibitive in practical applications as each batch of new documents would require retraining the model. When TextGCN and other graph-based methods are adapted for inductive learning, where the test set is unseen, they achieve notably lower scores [52]. Note that all previously described bag-of-words and sequence-based models fall in the inductive category and can be applied to new documents.

We briefly discuss further selected graph-based methods. HeteGCN combined ideas from Predictive Text Embedding [164] and TextGCN and splits the adjacency matrix into its word-document and word-word sub-matrices and fuses the different layers' representations when needed. TensorGCN explores and combines multiple different ways of converting the text into a graph, such as a semantic graph created with an LSTM, a syntactic graph created by dependency parsing, and a sequential graph based on word co-occurrence. HyperGAT combines graph attention [165] with the concept of hyperedges based on sequential structure and topic models [166]. AGGNN [161] focuses on text pooling mechanism along with gated graph sequence neural networks [167]. DADGNN is a graph-based approach that uses attention diffusion and decoupling techniques for tackling the over-smoothing problem of the GNN and building deeper models. Lastly, STGCN tackles short text classification by building upon ideas from TextGCN and adding word-topic and document-topic edges from a topic model, similar to HyperGAT. The authors also experimented with combining STGCN with a BiLSTM and a BERT model. In their experiments, the combination STGCN+BERT+BiLSTM gave the best results, while pure STGCN fell behind pure BERT. MHGAT [168] follows a different approach and captures word order by adding position-specific hyperedges to the graph to be processed by a graph attention network. These position edges are obtained through a sine/cosine transformation, following a similar strategy as in the Vaswani transformer's positional encoding [136].

Particularly between 2022 and 2024, numerous new graph-based models have been published. Many of them use BERT in conjunction *with an explicit graph encoder*, usually a graph neural network, such as in BertGCN [169], CTGCN [170], ILGCN [171], TSW-GNN [172], and ConTextING [133]. They differ from the other GNN-based methods as the graph is not computed based on word co-occurrences but BERT's subword tokens. Further graph-based methods are KGAT [173], InducT-GCN [162], TextSSL [174], GLTC [175], and others. A recent survey on GNNs for text classification was performed by Wang, Ding, and Han [60]. Moreover, Bugueño and de Melo [62] compare different initial document representations including Word2vec [122], GloVe [123], and frozen BERT embeddings and use them in conjunction with graph neural networks. They employ frozen and fine-tuned BERT models as baselines for the categorical text classification task. Their results show that—on most datasets—graph neural networks hardly compete with a fine-tuned BERT.

B. Hierarchy-based Methods

In addition to using text-induced graphs as described above, the dataset's classes may be organized into a graph structure. This is the case in hierarchical text classification, where each document should be annotated with a set of labels rather than a single class label [1] and the classes are organized along a taxonomy. The taxonomic hierarchy of labels is typically modeled as a tree or a directed acyclic graph [38], [176], [177]. The goal is to predict multiple class labels that correspond to one or more nodes in the hierarchy.

Following the taxonomic hierarchy to the root, the classes become more general (broader), while going towards the leaves, the classes become more specific (narrower). The documents are typically annotated with some specific classes in the taxonomy. In hierarchical text classification, this taxonomy is often used to *enrich* the gold standard [38]. This means that all vertices along the entire path from the root to the assigned classes are added as ground truth. The hierarchy-based text classifier HiAGM [38] and its follow-up works, such as HGCLR [18] and HBGL [17], rely on this enrichment.

The enrichment affects both the training and evaluation of hierarchy-aware methods. After this enrichment, the dataset consists of a set of documents $\boldsymbol{X}$. Each document is annotated with multiple labels, typically modeled as a label indicator matrix $\hat{\boldsymbol{Y}}$, and a hierarchy of classes $\boldsymbol{H}$. Then, the goal is to learn a function $\boldsymbol{x}, \boldsymbol{H} \mapsto \hat{\boldsymbol{y}}$ that maps the current document $\boldsymbol{x}$ to a set of enriched labels $\hat{\boldsymbol{y}}$, while also taking into account the label hierarchy $\boldsymbol{H}$. The hierarchy-based methods make use of an explicit graph encoder to represent the taxonomy and to exploit it in the model architecture to classify the text.

We briefly discuss the selected methods. HiAGM is a hierarchical text classifier that models the hierarchy as a directed graph along with hierarchy-aware structure encoders [38]. It comes in two variants: HiAGM-LA, which is a multi-label attention model that uses an inductive approach. HiAGM-TP is a text feature propagation model that uses a deductive approach to extract hierarchy-aware text features. It uses a GNN-based encoder to obtain a representation for each class and compare it with a Tree-LSTM representation of the text. In early 2021, several other hierarchical label-based attention models were published. For example, HLAN [178], LA-HCN [179], RLHR [180], and the weakly-supervised TaxoClass [176]. Other hierarchy-based methods are [181]–[183].

Further methods that use BERT for text encoding and a graph neural network for the hierarchy are BERT+HiMATCH [184], as well as HGCLR. The latter uses contrastive learning to align text and graph representations. K-HTC combines BERT as a text encoder with a knowledge graph based on entities relevant to the classification task [185].

The hierarchy-aware and label-balanced (HALB) model extends HGCLR by replacing the classification with asymmetric loss and adding another loss for separating samples with similar representation but different labels [186]. The Hierarchy-aware Information Lossless contrastive Learning (HILL) model uses BERT as a text encoder and a graph encoder together with a hierarchy-aware contrastive loss [187]. HGBL (note, different from HBGL described earlier) is another model based on contrastive learning on the label and text features using a graph-encoder and BERT as text encoder [188]. RADAr uses an encoder-decoder architecture for hierarchical text classification without requiring a graph encoder [40]. It uses RoBERTa as a text encoder and a custom autoregressive decoder with two transformer layers to generate the labels in level-wise order. HYDRA simplifies this architecture and relies solely on RoBERTa as the text encoder, together with level-wise losses to model the taxonomy [39].

The methods discussed so far rely on a small encoder-only language model. Retrieval-style ICL is an approach for hierarchical text classification using a large-language model and few-shot in-context learning [36].

VI. Experimental Apparatus

We introduce the benchmark datasets we identified for single-label, multi-label, and hierarchical text classification. We provide an overview of the models considered from the different families of text classification approaches. While the vast majority of results are taken from the literature, we also conduct our own runs. For these, we describe our procedure, choice of hyperparameters and their optimization, and evaluation measures.

A. Datasets

Our quantitative comparison focuses on topic classification, while including popular sentiment analysis datasets as control: MovieReviews for single-label classification, and GoEmotions for multi-label classification.

a) Single-label Datasets: We use the benchmark datasets 20ng, R8, R52, ohsumed, and MR with their standard train-test splits. Twenty Newsgroups (20ng)[1] (bydate version) contains long posts categorized into 20 newsgroups. R8 and R52 are subsets of the R21578 [189] news dataset with 8 and 52 classes, respectively. It is a collection of documents that appeared on Reuters newswire in 1987. Ohsumed[2] is a corpus of medical abstracts from the MEDLINE database that are categorized into diseases (one per abstract). Movie Reviews (MR)[3] [190], split by Tang et al. [164], is a binary sentiment analysis dataset on sentence level. Table I shows the dataset characteristics.

[1] http://qwone.com/~jason/20Newsgroups/
[2] http://disi.unitn.it/moschitti/corpora.htm
[3] https://www.cs.cornell.edu/people/pabo/movie-review-data/

TABLE I
Characteristics of the single-label text classification datasets. We show the number of documents N and the standard train-test split. #C is the number of classes. Finally, we report the documents' average length and standard deviation.

Dataset	N	#Train	#Test	#C	Avg. length
20ng	18,846	11,314	7,532	20	551 ± 2,047
R8	7,674	5,485	2,189	8	119 ± 128
R52	9,100	6,532	2,568	52	126 ± 133
ohsumed	7,400	3,357	4,043	23	285 ± 123
MR	10,662	7,108	3,554	2	25 ± 11

b) Multi-label and Hierarchical Datasets: Table II shows the characteristics of the multi-label and hierarchical datasets. Reuters-21578 (R21578) [189] is a popular dataset for multi-label classification. We use the train-test split from NLTK.[4] The labels in R21578 are not hierarchically organized. RCV1-V2 is a newer version of the R21578 dataset containing a much larger amount of hierarchically categorized newswire stories. For RCV1-V2, we use the train-test split proposed by Lewis et al. [191]. EconBiz [67] is a dataset containing scientific papers in economics. It provides the titles of a metadata export as well as the full text of papers up to 2017. We use the split provided by the authors for comparability. GoEmotions is a corpus of comments extracted from Reddit, with human annotations to 27 emotion categories [192]. We use the same train-test split as in the original paper. GoEmotions does not have a hierarchical label structure. Amazon-531 [193] contains 49,145 product reviews and a three-level class taxonomy consisting of 531 classes. DBPedia-298 [194] includes 245,832 Wikipedia articles and a three-level class taxonomy with 298 classes. For Amazon-531 and DBPedia-298, we use the same train-test split as in TaxoClass [176]. NYT AC [195] contains New York Times articles from 1987 to 2007. We use the train-validation-test split from HiAGM [38]. The Web of Science (WOS) dataset contains 46,985 scientific papers [196], each with exactly one parent and one child label. In the hierarchical datasets (Hier.=Y), each label set includes the more general labels along the path up to the root of the hierarchy.

B. Methods and Complementing Experiments

We build our quantitative comparison on existing studies such as Ding et al. [157], Ragesh et al. [52], Li et al. [197], and Galke et al. [21].

a) BoW-based Methods: For the methods based on Bag of Words (BoW), we rely on the study by Galke et al. [21], who have evaluated various variants of an MLP operating on a bag-of-words. These include an MLP with one wide hidden layer and two hidden layers and different text representations as input, such as TF-IDF weighting, and using pre-trained word embeddings such as GloVe. We further list the numbers for fastText, SWEM, and logistic regression from Ding et al. [157]. Motivated by [106], we complement these numbers by running SVMs on unigram and trigram features to include strong classical machine learning methods [68].

[4] https://www.nltk.org/book/ch02.html

TABLE II
CHARACTERISTICS OF THE MULTI-LABEL AND HIERARCHICAL CLASSIFICATION DATASETS. WE SHOW THE SAME STATISTICS FOR THE SINGLE-LABEL DATASETS. IN ADDITION, WE REPORT THE AVERAGE NUMBER OF CLASS LABELS PER DOCUMENT AND WHETHER THE DATASET COMES WITH A HIERARCHY (HIER.).

Dataset	Hier.	N	#Train	#Test	#C	Labels per doc.
R21578	N	10,788	7,769	3,019	90	1.24 ± 0.75
RCV1-V2	Y	804,414	23,149	781,265	103	3.24 ± 1.40
EconBiz	Y	1,064,634	994,015	70,619	5,661	4.36 ± 1.90
GoEmotions	N	48,837	43,410	5,427	28	1.18 ± 0.42
Amazon-531	Y	49,145	29,487	19,658	531	2.93 ± 0.26
DBPedia-298	Y	245,832	196,665	49,197	298	3.00 ± 0.00
NYT AC	Y	36,471	29,179	7,292	166	7.59 ± 5.61
WOS	Y	46,985	37,588	9,397	141	2.00 ± 0.00

b) Recurrent and Convolutional Neural Networks: We include scores of sequence-based models such as LSTMs [134], [157] and CNNs [198], [199]. In contrast to the BoW-based models, these consider the sequence of the textual input and exploit this to train the classifier.

c) Small Language Models: We run experiments with various pre-trained language models. The numbers reported in the literature for BERT applied to the same datasets vary widely across papers. We fine-tune BERT and some of the most popular encoder-only language models, including DistilBERT, RoBERTa, DeBERTa, ALBERTv2, and ERNIE 2.0 ourselves. Furthermore, we conducted runs with the encoder-decoder models BART and T5, as there were no numbers available in the literature.

d) Large Language Models: We found numbers for LLMs including GPT-3.5 (`text-davinci-003`) with the CARP prompting strategy [25], a sophisticated few-shot learning technique that relies on sampling examples based on RoBERTa-large embeddings (R.S. variant) and voting among multiple runs. Note that in-context learning evaluations of LLMs, such as CARP, do not make use of the full training set but only a few examples. A different strategy we include here is to fine-tune the parameters of LLMs for text classification, as done in RGPT [41]. Specifically, RGPT consists of an ensemble of fine-tuned Llama-2 models [152].

e) Synthetic Text-Graph Methods: We review both transductive and inductive graph-based methods for text classification. These include TextGCN along with its successors HeteGCN, TensorGCN, HyperGAT, DADGNN, Simplified GCN (SGC) [200], and many others. Ragesh et al. [52] evaluated a variant of TextGCN that is capable of inductive learning, which we include in our results.

f) Hierarchy-based Methods: For hierarchy-based text classification, we report results for models that use graph encoders such as BERT+HiMatch, HiAGM, HGCLR, and HBGL. We report models that do not use a graph encoder, e. g., the encoder-decoder-based Seq2Tree and RADAr, as well as HYDRA, which uses layer-wise losses on RoBERTa. For comparison, we run experiments with the BoW-based method WideMLP [21], which considers the classes as a set. More training is needed for transformer-based models on hierarchical datasets than on multi-label datasets [39].

C. Procedure

We distinguish between single-label and multi-label text classification settings. We apply standard train-test splits unless there is no default split provided (see Section VI-A).

We consider an inductive setting, where the test documents remain unseen until test time, i. e., they are not available for training or preprocessing. The distinction only matters for graph-based approaches because BoW-based and sequence-based models are usually inductive.

To avoid bias in the comparability of the results, we carefully checked all relevant parameters, such as the train/test splits, the number of classes in the datasets, whether datasets have been pre-processed to make the task substantially easier, and the evaluation metrics (see methodology in Section II).

We repeat all of our own experiments five times with a different random initialization of the parameters and report the mean and standard deviation of these five runs. To tune the hyperparameters of the multi-label classification models, we choose randomly 20% of the train set as a validation set. Below, we provide a detailed description of the hyperparameter optimization and evaluation procedures for the models that we have run ourselves.

D. Hyperparameters

Generally, we rely on the hyperparameter values and hyperparameter optimization strategies reported by the original works. In cases where we run experiments, we describe below how we set the hyperparameter values for the single-label, multi-label, and hierarchical text classification tasks. For each case, we follow the families of classification approaches.

a) Single-label Case: For the unigram and word-trigram SVM models, we first transform the features via TF-IDF and then employ a linear support vector classifier with hinge loss and default hyperparameters (l2 penalty, regularization strength $C = 1$).

For *fine-tuning the small language models* such as BERT-base, DistilBERT, RoBERTa-base, DeBERTa-base, ERNIE, and ALBERT, we adopt the fine-tuning strategy of Galke and Scherp [21]: We fine-tune each model for 10 epochs with a linearly decaying learning rate. We conduct a manual search among learning rates between 10^{-5} and $5 \cdot 10^{-5}$, as common for fine-tuning pre-trained language models [20], [22]. Our search range for batch sizes was between 4 and 128 – which in turn control the number of optimizer steps. Specifically,

we use $\text{lr} = 4.5 \cdot 10^{-5}$ for DistilBERT, $4 \cdot 10^{-5}$ and for RoBERTa, $2 \cdot 10^{-5}$ for DeBERTa, $2.5 \cdot 10^{-5}$ for ERNIE, and $1 \cdot 10^{-5}$ for ALBERT and BERT-large. The effective batch sizes (i.e., taking gradient accumulation into account) are 128 for DistilBERT, 16 for DeBERTa and BERT-large, and 32 for RoBERTa, ERNIE, and ALBERT. The transformers' output sequence is pooled according to the respective model's suggested strategy. For example, for BERT and most BERT-like models, the final representation of the first token is used (the inserted special CLS token) and fed into an MLP module. We use the uncased versions where available, e. g., BERT, and the cased variants of later models, e. g., RoBERTa, which no longer provide a separate uncased variant. We fine-tune BART and T5 for 10 epochs with a learning rate of $5 \cdot 10^{-5}$. For this, we use the discriminative variant, see Section IV-C, i. e., attach a newly initialized classifier head that processes the last token's representation and fine-tune all model parameters against a categorical cross-entropy loss.

b) Multi-label Case: For *training the WideMLP* in the multi-label case, we tune the hyperparameters using a manual search on the R21578 dataset with learning rates between 10^{-3} and 10^{-1} and batch sizes between 8 and 256. We noticed that higher learning rates are beneficial combined with a TF-IDF input representation and 100 epochs of training. We thus train the MLPs consistently with a learning rate of 10^{-1} for all datasets. We increase the batch size according to the dataset size in order to limit the overall training time: For the smaller datasets (R21578, GoEmotions, Amazon-531, NYT AC, RCV1-V2), we use a batch size of 8. For DBPedia-298, the batch size is 32, and for EconBiz, it is 256.

All models are trained with Adam [201] to minimize binary cross-entropy loss. At test time, class labels are assigned depending on a threshold on the class-specific output. It is common to threshold the sigmoidal output units at values of $\lambda = 0.5$ [202], [203]. However, Galke et al. [68] had found that a smaller threshold such as $\lambda = 0.2$ can be advantageous, especially in setups with an imbalanced label distribution. In pre-experiments with WideMLP, we tested different thresholds from $\lambda = 0.5$ to $\lambda = 0.1$ (0.1 steps) and experienced similar results as reported in Galke et al. [68].

For the *encoder-only models*, i. e., the variants of BERT, we use a manual search to find the best hyperparameters using a search range in learning rate between 10^{-5} and $5 \cdot 10^{-5}$, batch sizes between 4 and 128, and epochs in $\{5, 10, 15\}$. We again fine-tune all parameters of the model, as we do for single-label classification, with Adam to minimize binary cross-entropy loss. In the pre-experiments with R21578, we found that small batch sizes are preferable for fine-tuning these models. We used a learning rate of $5 \cdot 10^{-5}$ with a batch size of 4 for all data sets. We fine-tune the models on the datasets for either 5 or 15 epochs for multi-label training, following [21]. DBPedia-298 and GoEmotions had the best results with 5 epochs as the validation loss increased in subsequent iterations. The encoder models employ a threshold of $\lambda = 0.5$ after pre-experiments with 0.2 and 0.5.

For the encoder-decoder models BART and T5, we use Adam to optimize all parameters (incl. classifier head) against binary cross-entropy with a learning rate of $5 \cdot 10^{-5}$. We use the best identified number of epochs and batch sizes as determined for the encoder-only models, and we again test thresholds for class assignment $\lambda = 0.2$ and 0.5, out of which the 0.2 threshold led to better results.

c) Hierarchical Case: The runs on the datasets NYT and RCV1-v2 use the same hyperparameters as described for the multi-label case above. For WOS, we train WideMLP for 100 epochs with a learning rate of $3 \cdot 10^{-1}$ and a batch size of 128. The discriminative variants of BART and T5 on WOS were trained with a linearly decaying learning rate of $2 \cdot 10^{-5}$, weight decay of 0.1, and gradient clipping to 1.0. For other hierarchy-based methods, we rely on numbers from the literature.

E. Measures

We report accuracy as the evaluation metric for single-label datasets. Note that the accuracy is equivalent to Micro-F1 in single-label classification [21]. We report the mean accuracy and standard deviation (SD) over five runs for neural network methods, which rely on random initialization and other noise sources during training, such as dropout. For those models where we rely on numbers from the literature, we check if multiple runs are reported and include the corresponding information in our report. Note that the exact number of runs may differ from paper to paper. Most papers report five runs (if they have multiple runs), but others report ten runs.

For the multi-label datasets, we follow Galke et al. [68] and report the sample-based F1 measure. We chose this sample-based evaluation measure because it reflects the classification quality of each document separately [67]. The sample-based F1 measure is calculated by the harmonic mean of precision and recall for each example individually, and then these scores are averaged. For hierarchical text classification, we report globally averaged Micro-F1 and class-averaged Macro-F1, as common in the hierarchical text classification literature.

VII. Quantitative Comparison

We present the results of our quantitative comparison, starting with single-label text classification. Subsequently, we present our results of a sensitivity analysis of transformer models regarding the fine-tuning learning rate. This is followed by the results on multi-label and hierarchical text classification. We bold the best numbers and underline the second best per model family.

A. Single-label Text Classification

Table III shows the results of the single-label text classification on the five datasets. We see that sequence-based transformers are generally the best methods, followed by the graph-based methods and BoW methods.

Among the BoW-based methods, the SVM variants are the strongest on ohsumed and MR, followed by WideMLP and SWEM. The CFE-IterativeAdditive with its use of label matching to compute adapted global IDF weights TF-ICF is the best BoW model on 20ng, R8, and R52. The strength of TF-IDF weighting is also shown by the Trigram SVM with TF-IDF and WideMLP with TF-IDF. We further observe that

TABLE III

RESULTS FOR THE SINGLE-LABEL TEXT CLASSIFICATION DATASETS. FOR OUR EXPERIMENTS, WE REPORT THE MEAN ACCURACY AND STANDARD DEVIATION (SD) OVER FIVE RUNS. FOR NUMBERS FROM THE LITERATURE, WE REPORT THE SD IF AVAILABLE. GPT-3.5 METHODS USE 16 EXAMPLES PER CLASS FOR IN-CONTEXT LEARNING, AND THE (R.S)-VARIANT USES A SUP-SIMCSE-ROBERTA-LARGE SAMPLER TO SELECT THESE EXAMPLES. COLUMN "PROVENANCE" REPORTS THE SOURCE. D. F. IS SHORT FOR THE USE OF A DIFFERENT VARIANT OF THE DATASET OR A DIFFERENT SPLIT, AND THUS, THE NUMBER IS OMITTED TO ENSURE COMPARABILITY.

Inductive Setting	**20ng**	**R8**	**R52**	**ohsumed**	**MR**	**Provenance**
BoW-based Methods						
Logistic regression with TF-IDF	83.70	93.33	90.65	61.14	76.28	[52]
Unigram SVM with TF-IDF	83.44	97.49	94.70	67.40	76.36	our experiment
Trigram SVM with TF-IDF	83.39	97.21	93.85	**69.30**	**77.35**	our experiment
XGBoost with TF-IDF	73.29	94.75	88.82	58.27	64.46	our experiment
WideMLP with TF-IDF	84.20 0.16	97.08 0.16	93.67 0.23	66.06 0.29	76.32 0.17	[21]
WideMLP	83.31 0.22	97.27 0.12	93.89 0.16	63.95 0.13	76.72 0.26	[21]
WideMLP-2	81.02 0.23	96.61 1.22	93.98 0.23	61.71 0.33	75.91 0.51	[21]
GloVe+WideMLP	76.80 0.11	96.44 0.08	93.58 0.06	61.36 0.22	75.96 0.17	[21]
GloVe+WideMLP-2	76.33 0.18	96.50 0.14	93.19 0.11	61.65 0.27	75.72 0.45	[21]
SWEM	85.16 0.29	95.32 0.26	92.94 0.24	63.12 0.55	76.65 0.63	[157]
fastText	79.38 0.30	96.13 0.21	92.81 0.09	57.70 0.49	75.14 0.20	[157]
CFE-IterativeAdditive	**85.51** 0.04	**97.94** 0.02	**95.13** 0.04	68.90 0.02	—	[121]
Sequence-based Methods						
CNN+GloVe	82.15	95.71	87.59	58.44	77.75	[133]
CNN-non-static	—	—	—	—	81.5	[199]
Word2Vec+CNN	—	—	—	—	81.24	[198]
GloVe+CNN	—	—	—	—	81.03	[198]
LSTM w/ pre-training	75.43 1.72	96.09 0.19	90.48 0.86	51.10 1.50	77.33 0.89	[157]
Bi-LSTM GloVe embeddings	—	96.31	90.54	—	77.68	[134]
GPT-3.5 finetuning via OpenAI API	—	—	95.27 0.55	51.84 0.45	—	[197]
Bloom-7.1B 4-bit LoRA finetuning	—	—	—	67.54 0.6	—	[197]
Llama2-7B 4-bit LoRA finetuning	—	—	—	67.66 0.72	—	[197]
Llama3-8B 4-bit LoRA finetuning	—	—	—	68.02 0.26	—	[197]
GPT-3.5 16-shot	—	91.58	91.56	—	89.15	[25]
GPT-3.5+CoT 16-shot	—	92.49	92.03	—	89.91	[25]
GPT-3.5+CARP 16-shot	—	97.60	96.19	—	90.03	[25]
GPT-3.5 (R.S.) 16-shot	—	95.57	95.79	—	90.90	[25]
GPT-3.5+CoT (R.S.) 16-shot	—	95.59	95.89	—	90.17	[25]
GPT-3.5+CARP+vote (R.S.) 16-shot	—	98.78	96.95	—	**92.39**	[25]
RGPT w. RoBERTa-base	—	—	—	74.70	d. f.	[41]
RGPT w. k=7 Llama 2 + LoRA	—	—	—	**77.41**	d. f.	[41]
QLFRw. LLaMA2-13B + LoRA 20-shot	—	—	—	61.10	81.70	[204]
LLMEmbed BERT+RoBERTa+Llama embeddings	—	98.22	95.68	—	d. f.	[32]
BERT-base	87.21 0.18	98.03 0.24	96.17 0.33	71.46 0.54	86.61 0.38	[21]
BERT-large	85.83 0.64	97.98 0.29	96.41 0.28	72.69 0.63	88.22 0.21	our experiment
RoBERTa-base	86.80 0.51	98.19 0.18	97.13 0.10	75.08 0.42	88.68 0.29	our experiment
DeBERTa-base	87.60 0.45	98.30 0.20	97.10 0.13	75.94 0.33	89.98 0.26	our experiment
DistilBERT-base	86.90 0.04	97.93 0.11	96.89 0.12	71.65 0.38	85.11 0.25	our experiment
ERNIE 2.0	87.79 0.29	97.95 0.16	96.96 0.23	73.33 0.30	89.19 0.24	our experiment
ALBERTv2	82.08 0.30	97.88 0.22	94.95 0.20	62.31 2.11	86.28 0.21	our experiment
AM-BERT BERT-base	89.03	98.43	97.17	73.47	86.83	[145]
AM-RoBERTa RoBERTa-base	**90.32**	**98.97**	**98.12**	73.89	89.75	[145]
BART-base discriminative	87.24 0.17	98.14 0.15	96.99 0.15	75.38 0.21	87.41 0.45	our experiment
T5-base discriminative	85.90 0.23	97.98 0.05	95.16 0.13	72.14 0.29	90.10 0.38	our experiment
LFTC (compression w. 1-NN)	81.4	96.5	90.6	43.5	—	[205]
Graph-based Methods						
Text-level GNN	—	97.8 0.2	94.6 0.3	69.4 0.6	—	[163]
TextING-M	—	98.13 0.31	95.68 0.35	70.84 0.52	80.19 0.31	[16]
TextGCN	80.88 0.54	94.00 0.40	89.39 0.38	56.32 1.36	74.60 0.43	[52]
HeteGCN	84.59 0.14	97.17 0.33	93.89 0.45	63.79 0.80	75.62 0.26	[52]
HyperGAT-ind	84.63	97.03	94.55	67.33	77.08 0.27	[133]
DADGNN	—	98.15 0.16	95.16 0.22	—	78.64 0.29	[159]
SGNN	—	98.09	95.46	—	80.58	[134]
ESGNN	—	98.23	95.72	—	80.93	[134]
C-BERT (ESGNN+BERT) BERT-base	—	**98.28**	**96.52**	—	86.06	[134]
ConTextING-RoBERTa RoBERTa-base	85.00	98.13	96.40	72.53	**89.43**	[133]
TextSSL	85.26 0.28	97.81 0.14	95.48 0.26	70.59 0.38	79.74 0.19	[174]
GLTC	—	98.17	95.77	71.82	80.29	[175]
InducT-GCN	—	96.53 0.17	93.23 0.15	—	—	[162]
MHGAT	**92.68** 0.30	97.65 0.47	94.78 0.37	**72.88** 0.84	78.09 0.73	[168]

pure BoW and TF-IDF-weighted BoW yield better results than approaches that exploit pre-trained word embeddings such as GloVe-MLP, fastText, and SWEM.

Regarding the sequence-based methods, a GPT 3.5 with 16 shots and CARP and voting achieves the best results on MR. It requires a fine-tuned RoBERTa, which is used to select the most relevant shots. RGPT achieves the highest score on ohsumed with a sequence of seven fine-tuned Llama 2 models and low-rank adaptation. AM-RoBERTa, using two additional attention matrices for keywords and term dependencies, achieves the best results on 20ng, R8, and R52. DeBERTa-base attains the highest scores among the encoder-only models. The margin to a BERT-base model is most notable on ohsumed (75.9 vs. 71.5) and on MR (90.0 vs. 86.6). BERT-large improves the scores over BERT-base by a small 1 point for the ohsumed and MR datasets, but the inverse of a performance decrease of 1 point is recorded for 20ng. For R8 and R52, both BERT-base and BERT-large achieve about the same performance. BART-base is a strong single-label classifier, outperforming BERT-base on all single-label datasets and BERT-large on four datasets. T5-base generally falls behind BART-base, except for the MR datasets, where it has a 2.69-point advantage. BART and T5 cannot beat DeBERTa, except for MR, where T5 shows a marginal improvement of 0.12 points over DeBERTa.

The graph-based MHGAT obtains the highest scores on the 20ng dataset among all method families. However, numbers for GPT-3.5 with CARP and RGPT were not available for this dataset for comparison. With the help of a pre-trained language model, models like C-BERT and ConTextING-RoBERTa attain higher scores on R8, R52, and MR. The largest difference is found on the MR sentiment analysis dataset, where ConTextING-RoBERTa reaches 89.43 compared to 77.08 of HyperGAT-ind. The best-performing graph-based model not using a pre-trained language model is TextSSL, closely followed by HyperGAT. TextGCN is outperformed by a BoW-based logistic regression on four out of five datasets [52].

The best pure graph-based method TextSSL is outperformed by T5-base by 10 points on the MR dataset. Comparing DeBERTa with the best graph-based method ConTextING-RoBERTa, there is still superiority of the pure transformer, but the margin is smaller.

B. Sensitivity Analysis to Fine-tuning Learning Rate for Single-label Classification

Analyzing the results on single-label datasets reported in the literature, we noticed that the performances of BERT and other transformer models on the same datasets differ from paper to paper. To shed light on the differences between BERT results on the same datasets, we provide an overview in Table IV and conduct additional runs with different learning rates. In our additional runs, we kept preprocessing and other parameters like batch size and tuning effort the same.

We observe that there are substantial differences between the supposedly same BERT models applied on the same datasets reported in the literature. For BERT-base, the difference is, in many cases, 2 and 3 points on the 20ng and ohsumed datasets, respectively. For RoBERTa, we even observe deviations of more than 3 points on 20ng and 5 points on ohsumed despite using the same learning rate.

Some of the reported numbers for fine-tuned BERT models are even far behind the others. For example, Yin et al. [175] report BERT-base results that are more than ten points behind the other papers on MR and ohsumed. We hypothesize that this discrepancy is caused by a suboptimal choice of hyperparameters, e. g., a too-high learning rate, which are unfortunately not provided.

On the R8, R52, and MR datasets, the results differ by not more than 1 point. Remarkably, DistilBERT is quite sensitive to a small change in the learning rate by only $0.5 \cdot 10^{-5}$. The difference is more than 1 point on R52 and even 2 points on ohsumed.

C. Multi-label Text Classification

For multi-label classification, we show the sample-based F1 scores in Table V. We argue that the sample-based F1 represents real-world applications where each document needs to be annotated one document after the other, such as in subject indexing in digital libraries [67], [68].

The T5-base model is the strongest or second-strongest of the sequence-based models for multi-label datasets, except GoEmotions. This differs from the results on the single-label datasets, where T5-base was behind encoder-only models, except for MR.

A WideMLP with TF-IDF is falling behind the sequence-based models. For some datasets, like DBpedia, the differences among the sequence-based methods are very small, while for others, a difference of up to three points can be observed.

Many of the multi-label datasets are hierarchical, i. e., have hierarchically organized labels. We report the use of hierarchy-based methods for three multi-label classification datasets with hierarchy in the next section. Furthermore, we report results involving LLMs in the next section, too.

D. Hierarchical Text Classification

Table VI shows results for Micro-F1 and Macro-F1 metrics for the three hierarchical datasets: Web of Science (WOS) [196], NYT, and RCV1-V2 (see Section VI-A).

Sequence-based models perform better than the hierarchy-based methods, except for Micro-F1 on WOS. The BoW-based methods clearly fall behind, most notably on the more difficult-to-improve F1-Macro measure.

As with single-label and multi-label classification, TF-IDF with BoW is stronger than pure BoW. In terms of the sequence-based methods, the best performer depends on the dataset and measure. But all of them are based on an encoder-only model like HBGL or an encoder-decoder like Seq2Tree. The LLM-based Retrieval-style ICL uses a 16-shot in-context learning, clearly falling behind. A fine-tuned and unmasked Llama-7B cannot reach the performance of smaller language models like encoder-only variants. The difference is particularly notable on F1-Macro.

Regarding the hierarchy-based methods, the strongest method is HyILR. It is followed by other hierarchy-based

TABLE IV
COMPARISON OF DIFFERENT TRANSFORMER MODELS AND HYPERPARAMETER SETTINGS. WE REPORT MEAN ACCURACY AND STANDARD DEVIATION OVER FIVE RUNS ON THE SINGLE-LABEL TEXT CLASSIFICATION DATASETS (INDUCTIVE). COLUMN "PROVENANCE" REPORTS THE SOURCE. N/P REFERS TO THE CASE WHERE THE PAPER (OR POTENTIAL SUPPLEMENTARY MATERIALS) DID NOT PROVIDE INFORMATION ABOUT THE LEARNING RATE.

Inductive Setting	**20ng**	**R8**	**R52**	**ohsumed**	**MR**	**Provenance**
BERT-base (lr $= 5 \cdot 10^{-5}$)	87.21 $_{0.18}$	98.03 $_{0.24}$	96.17 $_{0.33}$	71.46 $_{0.54}$	86.61 $_{0.38}$	our experiment
BERT-base (lr $= 5 \cdot 10^{-5}$)	—	—	95.55 $_{0.36}$	65.71 $_{0.30}$	—	[197]
BERT-base (lr $= 3.5 \cdot 10^{-5}$)	87.31 $_{0.21}$	98.19 $_{0.12}$	97.13 $_{0.16}$	73.54 $_{0.45}$	86.86 $_{0.10}$	our experiment
BERT-base (lr $= 2 \cdot 10^{-5}$)[1)]	85.20	97.73	96.22	70.53	85.71	[145]
BERT-base (lr $= 1 \cdot 10^{-5}$)	84.54	97.26	96.26	68.74	85.88	[133]
BERT-base (lr $= 1 \cdot 10^{-5}$)	85.3	97.8	96.4	70.5	85.7	[169]
BERT-base (lr $=$ N/P)[1)]	—	96.78	91.35	60.46	76.13	[175]
BERT-base (lr $=$ N/P)	—	98.2	—	—	85.7	[160]
BERT-base (lr $=$ N/P)	83.1	—	—	—	—	[206]
DistilBERT-base (lr $= 5 \cdot 10^{-5}$)	86.24 $_{0.26}$	97.89 $_{0.15}$	95.34 $_{0.08}$	69.08 $_{0.60}$	85.10 $_{0.33}$	our experiment
DistilBERT-base (lr $= 4.5 \cdot 10^{-5}$)	86.90 $_{0.04}$	97.93 $_{0.11}$	96.89 $_{0.12}$	71.65 $_{0.38}$	85.11 $_{0.25}$	our experiment
RoBERTa-base (lr $= 4 \cdot 10^{-5}$)	86.80 $_{0.51}$	98.19 $_{0.18}$	97.13 $_{0.10}$	75.08 $_{0.42}$	88.68 $_{0.29}$	our experiment
RoBERTa-base (lr $= 4 \cdot 10^{-5}$)	83.8	97.8	96.2	70.7	89.4	[169]
RoBERTa-base (lr $= 1 \cdot 10^{-5}$)	84.07	97.35	95.48	69.86	87.08	[133]
RoBERTa-base (lr $=$ N/P)[1)]	83.80	97.80	96.20	70.70	89.40	[145]

[1)] Authors applied special preprocessing of the input text.

TABLE V
RESULTS FOR THE INDUCTIVE MULTI-LABEL TEXT CLASSIFICATION DATASETS. WE REPORT THE SAMPLE-BASED F1 METRIC TO REFLECT HOW WELL THE CLASSIFIER PERFORMS ON AVERAGE PER A SET OF NEW DOCUMENTS. STANDARD DEVIATION ACROSS RUNS IS DENOTED IN BRACES.

Inductive Setting	**R21578**	**RCV1-V2**	**EconBiz**	**Amaz.**	**DBPedia**	**NYT**	**GoEmo.**
BoW-based methods							
WideMLP	80.41	69.92 $_{0.11}$	23.15	59.92	89.47	62.38 $_{0.27}$	37.13
TF-IDF WideMLP	**88.15**	**81.51** $_{0.03}$	**45.38**	**80.32**	**94.91**	**75.58** $_{0.09}$	**40.07**
Sequence-based methods							
BERT-base	92.21	88.16 $_{0.16}$	42.08	86.69	97.66	79.11 $_{0.22}$	54.18
BERT-large	92.23	88.83 $_{0.17}$	33.62	88.34	**97.69**	80.32 $_{0.39}$	54.02
DistilBERT	92.11	87.50 $_{0.11}$	39.41	87.47	97.58	79.18 $_{0.17}$	55.95
RoBERTa-base	90.85	88.62 $_{0.21}$	40.56	86.21	97.26	79.14 $_{0.55}$	54.64
DeBERTa-base	91.24	88.45 $_{0.19}$	41.43	**89.21**	97.65	79.95 $_{0.40}$	**56.51**
BART-base $_{\text{discriminative}}$	90.71 $_{0.35}$	86.62 $_{0.16}$	31.60	85.02 $_{0.57}$	96.44 $_{0.35}$	74.73 $_{0.60}$	52.64 $_{1.60}$
T5-base $_{\text{discriminative}}$	**92.72** $_{0.24}$	**89.13** $_{0.11}$	**45.63**	88.93 $_{0.33}$	97.68 $_{0.02}$	**80.79** $_{0.15}$	55.01 $_{0.62}$

methods as close runners-up, such as HILL, and is barely outperformed in two cases by HALB.

VIII. DISCUSSION

A. Key Findings

The results highlight that, overall, the strongest model variants are sequence-based language models. For single-label classification, encoder-only SLMs and encoder-decoder SLMs remain highly competitive to dedicated LLM pipelines. Data augmentation and carefully designed ensembles using LLMs yield the strongest overall results, while graph-based approaches provide only dataset-specific advantages. For multi-label classification, the encoder-decoder T5 model is the strongest approach (except for GoEmotions). In hierarchical datasets, one observes benefits from hierarchy-aware modeling in sequence-based models, which obtain the highest scores of all model families. Dedicated hierarchical methods using the given taxonomy generally fail to consistently outperform strong fine-tuned language models. Below we discuss the results in detail along the three tasks and per model families.

a) Single-label Classification: As shown in Table III, encoder-only SLMs show strong results. The graph-based MHGAT is the best method on the 20ng dataset. The LLM-based 16-shot ensemble GPT-3.5+CARP+voting using a fine-tuned RoBERTa-Large is the best on the MR dataset, while RGPT with an ensemble of k=7 Llama 2 models is the strongest model on ohsumed. Both LLM-based models were not evaluated on 20ng. We were not able to independently verify the results of MHGAT due to the lack of source code and the authors' accessibility, we can only infer from the reported ablations that the hypergraph-based positional embedding may be a beneficial factor for the 20ng dataset [168]. Regarding GPT-3.5+CARP+voting, the use of an ensemble and supervised fine-tuned RoBERTa-Large to select the 16-shot provides the advantages as shown in the results of the different GPT-3.5 model variants in Table III. For RGPT, it requires an ensemble of Llama 2 models to outperform the second-best model, a pure DeBERTa-base model, by 1.47 points.

TABLE VI
MICRO AND MACRO F1 SCORES WITH STANDARD DEVIATION (WHERE AVAILABLE) ACROSS FIVE RUNS FOR HIERARCHICAL MULTI-LABEL CLASSIFICATION ON THREE COMMON BENCHMARK DATASETS USING MICRO-F1 AND MACRO-F1 SCORES.

Model	**WOS** (Micro/Macro)	**NYT** (Micro/Macro)	**RCV1-V2** (Micro/Macro)	**Provenance**
BoW-based methods				
WideMLP	**73.36** $_{0.14}$ / **62.61** $_{0.23}$	57.18 $_{0.28}$ / 21.96 $_{0.19}$	68.31 $_{0.12}$ / 27.88 $_{0.49}$	our experiment
TF-IDF WideMLP	72.25 $_{0.08}$ / 59.85 $_{0.19}$	**74.53** $_{0.07}$ / **56.11** $_{0.16}$	**80.45** $_{0.02}$ / **53.27** $_{0.09}$	our experiment
Sequence-based methods				
BERT-base	86.19 $_{0.11}$ / 80.23 $_{0.20}$	79.07 $_{0.22}$ / 67.63 $_{0.42}$	86.38 $_{0.23}$ / 67.89 $_{1.42}$	[39]
BERT-base	85.63 / 79.07	78.24 / 65.62	85.65 / 67.02	[18]
BERT-base	86.26 / 80.58	—	86.26 / 67.35	[184]
BERT-large	86.66 $_{0.31}$ / 80.80 $_{0.38}$	80.69 $_{0.08}$ / 70.27 $_{0.39}$	87.40 $_{0.11}$ / 70.15 $_{0.39}$	[39]
DeBERTa-base	86.82 $_{0.11}$ / 80.85 $_{0.48}$	81.21 $_{0.17}$ / 71.33 $_{0.26}$	87.24 $_{0.18}$ / 69.66 $_{0.52}$	[39]
DeBERTaV3-base	86.58 $_{0.24}$ / 80.41 $_{0.69}$	79.96 $_{0.20}$ / 67.30 $_{0.23}$	86.77 $_{0.18}$ / 67.49 $_{1.87}$	[39]
RoBERTa-base	86.35 $_{0.15}$ / 80.16 $_{0.13}$	81.47 $_{0.15}$ / 71.45 $_{0.45}$	87.38 $_{0.17}$ / 68.60 $_{1.40}$	[39]
ModernBERT-base	86.15 $_{0.33}$ / 80.34 $_{0.22}$	79.83 $_{0.53}$ / 68.99 $_{0.54}$	86.20 $_{0.42}$ / 67.43 $_{0.66}$	[39]
HYDRA $_{\text{RoBERTa-base}}$	86.91 $_{0.18}$ / 81.22 $_{0.40}$	**81.96** $_{0.06}$ / 72.35 $_{0.22}$	**87.79** $_{0.11}$ / 70.08 $_{0.76}$	[39]
BART-base $_{\text{discriminative}}$	85.25 $_{0.30}$ / 77.22 $_{0.82}$	72.39 $_{0.61}$ / 52.04 $_{1.90}$	84.77 $_{0.21}$ / 63.63 $_{0.91}$	our experiment
T5-base $_{\text{discriminative}}$	79.21 $_{1.09}$ / 60.98 $_{3.41}$	79.06 $_{0.16}$ / 66.04 $_{0.10}$	87.41 $_{0.14}$ / 69.54 $_{0.56}$	our experiment
BART-base $_{\text{generative}}$	84.08 / 77.43	19.21 / 6.49	86.20 / 65.11	[40]
T5-base $_{\text{generative}}$	82.03 / 74.62	46.71 / 20.06	84.9 / 57.01	[40]
HiGen $_{\text{BART}}$	87.39 / 81.45	80.89 / 72.41	—	[207]
SGM	67.74 / 74.01	64.68 / **72.78**	71.85 / 35.29	[40]
SGM w. T5-base	85.83 / 80.79	—	84.39 / 65.09	[208]
Seq2Tree $_{\text{T5-base}}$	87.20 / **82.50**	—	86.88 / 70.01	[208]
RADAr $_{\text{RoBERTa-base}}$	87.17 $_{0.04}$ / 81.84 $_{0.08}$	79.84 $_{0.07}$ / 68.64 $_{0.28}$	87.23 $_{0.05}$ / 69.64 $_{0.12}$	[40]
HBGL $_{\text{BERT-base}}$	**87.36** / 82.00	80.47 / 70.19	87.23 / 71.07	[17]
HLPD $_{\text{T5-base}}$	87.05[1] / 81.84[1]	— / —	88.22 $_{0.12}$ 71.19 $_{0.55}$	[209]
LH-Mix $_{\text{BERT-base}}$	87.15 $_{0.14}$ / 81.82 $_{0.18}$	80.51 $_{0.15}$ / 71.03 $_{0.24}$	87.39 $_{0.12}$ / **71.49** $_{0.50}$	[210]
GPT 3.5 Turbo $_{\text{zero-shot}}$	—	—	51.35 $_{0.18}$ / 32.20 $_{0.30}$	[211]
Retrieval-style ICL $_{\text{16-shot Vicuna-7b + BERT-base}}$	81.12 $_{0.26}$ / 73.72 $_{0.17}$	—	—	[36]
unLlama-7B $_{\text{LLama 2 7B, LoRA-rank 12}}$	84.13 / 74.32	77.20 / 61.68	84.45 / 61.42	[39]
Hierarchy-based methods				
BERT+HiMatch $_{\text{BERT-base}}$	86.70 / 81.06	—	86.33 / 68.66	[184]
HiAGM-TP+GCN	85.82 / 80.28	74.97 / 60.83	83.96 / 63.35	[38]
HGCLR $_{\text{BERT-base}}$	87.11 / 81.20	78.86 / 67.96	86.49 / 68.31	[18]
HGBL $_{\text{BERT-base}}$	87.07 / 81.10	78.55 / 67.08	**87.55** / 68.10	[188]
HALB $_{\text{BERT-base}}$	87.45 / **82.04**	79.56 / 69.28	86.94 / 69.32	[186]
HILL $_{\text{BERT-base}}$	87.28 / 81.77	80.47 / 69.96	87.31 / 70.12	[187]
HE-AGCRCNN	—	—	77.8 / 51.3	[177]
K-HTC	87.29 / 81.69	—	—	[185]
HyILR $_{\text{BERT-base}}$	**87.48** $_{0.19}$ / 81.96 $_{0.22}$	**81.26** $_{0.23}$ / **70.71** $_{0.28}$	87.41 $_{0.23}$ / **71.20** $_{0.30}$	[212]
TaxMorph $_{\text{Sonnet-3.5}}$	— / 80.2[2]	— / —	— / —	[213]

[1] We report numbers directly obtained by the authors after some personal conversation.
[2] Use a 32.9k train, 7k val, and 7k test split.

Regarding the encoder-only models, BERT-large does not improve more than 1 point over BERT-base, despite having three times more parameters. On 20ng, the performance even drops by one point. Presumably, the capacity of BERT-base is already sufficient to tackle the single-label classification tasks, especially for the R8 and R52 datasets, and there are reports that BERT-large is harder to fine-tune on small datasets [20].

Augmentation improves the results and makes the SLM AM-RoBERTa model the overall strongest single-label classifier. A similar observation is made for the BoW methods, where the use of concept-based TF-ICF features with SVM in CFE-IterativeAdditive [121] improves the results in three datasets. Such an observation was also made by Galke et al. [68], who used CTF-IDF features for text classification.

b) Multi-label Classification: The strongest multi-label classifier is T5-base, outperforming the encoder-only RoBERTa and DeBERTa, in contrast to the general observation for the single-label datasets. An interesting observation can be made on the multi-label sentiment prediction dataset GoEmotions, where WideMLP achieves the worst performance across all models and the highest margin compared to BERT regarding all datasets. Similarly, BERT outperforms WideMLP on the single-label MR dataset with the largest margin compared to other datasets. However, WideMLP is a surprisingly strong model on the EconBiz dataset with the most number of labels. This may be due to the highly imbalanced (long tail) label distribution of the thousands of labels in the EconBiz dataset [67], which may be easier to reflect in a model trained from scratch than in a pre-trained model, such as BERT.

We assume that BoW-based MLP models are at a disadvantage in sentiment prediction compared to sequence-based models, where word order is relevant for the task. Note that most graph-based methods also discard word order when setting up the graph [21], except for models that combine the GNN with a sequence-based transformer model.

Since there are no sample-based F1 results in the literature for hierarchical multi-label datasets, we conducted an additional analysis (not shown in the table) and ran the best-performing HiAGM-TP variant with GCN as graph encoder [38] and the sequence-based HBGL [17] on the RCV1-V2 and NYT datasets. We use the optimized hyperparameters from the papers. The results are, HiAGM with a graph-encoder GCN obtained strong results (RCV1-V2: 85.51, Amazon: 89.05, DBPedia: 97.17, NYT: 76.57) but is outperformed by the best encoder-only models. It ran out of memory on EconBiz on a 40 GB RAM NVIDIA A100 HGX GPU. HBGL with BERT-base achieves on RCV1-V2 a sample-based F1 score of 88.76 and on NYT a score of 82.01. The former is outperformed by a T5-base, as shown in Table V, while the latter surpasses all encoder-only and encoder-decoder models considered, demonstrating that hierarchy-aware methods can be beneficial for hierarchical datasets. Note that HBGL and HiAGM cannot be applied to the flat multi-label datasets R21578 and GoEmotions as they do not have a label hierarchy.

Existing studies using LLMs focus on single-label datasets only, e. g., [28], [41]. But we were able to include LLMs for the hierarchical multi-label datasets, which we discuss below.

c) Hierarchical Classification: For hierarchical multi-label text classification, we conclude that sequence-based methods are best. Except for a slim margin of 0.12 for the hierarchy-based HyILR method regarding Micro-F1 on WOS, the best-performing models are sequence-based methods. Which sequence-based method performs best depends on the dataset and metric, but generally they are methods using fine-tuned SLMs. The hierarchy-based methods incorporating graph neural networks, e. g., to use a GNN to encode the class hierarchy as in HiAGM, fail to outperform the best of our tested language models.

A fine-tuned LLM unLlama with 7B parameters cannot outperform the SLMs. However, it should be noted that LLM-based methods with in-context learning are effective with small amounts of labeled data, which we further discuss below.

B. Fine-tuning SLMs vs. In-Context Learning in LLMs

An important discussion regarding model performance is the distinction between fine-tuning and in-context learning. When a sufficiently large training dataset is available, it is reported for single-label datasets that SLMs outperform the use of LLMs [28]. While SLMs *need* fine-tuning to obtain their good results, LLMs can also do in-context learning with few examples [28], which gives them an advantage when little training data is available. The LLM-based method CARP [25], i. e., the variant of GPT-3.5+CARP+vote (16-shot, RoBERTa sampler) requires a fine-tuned SLM (the RoBERTa model) for sampling the 16 most representative examples per class. This implies the need for contextualized embeddings from a fine-tuned encoder-only language model prior to prompting the decoder-only language model. While analyzing the importance of example selection is of scientific interest, practitioners should take into account that one could use the fine-tuned SLM model directly for the classification task. One would get results close to the fine-tuned SLM, while avoiding running an LLM at inference time.

The best-performing method on the single-label ohsumed dataset, RGPT, uses an ensemble of k=7 Llama 2 models, each of which is fine-tuned and applied sequentially. Each model has about 2.1 M effective trainable parameters (with LoRA rank = 8) [41], which shows that fine-tuning LLMs can be promising. Recent work supports the notion that fine-tuned models are generally advantageous over few-shot prompting when a minimum of labeled data is available [30], [31], [33], [214]. Pecher et al. [28] find for single-label datasets that in some cases already 100 labeled examples per class are sufficient for fine-tuned models to outperform in-context learning. However, the case for multi-label must be treated with more nuance as the sample complexity grows when a single example may be annotated with more than one label.

A key trick for effectively fine-tuning a LLM directly for a classification task may be to remove the constraint of the left-to-right attention mask [34]. However, experiments on fine-tuning unLlama with 7B parameters and LoRA on hierarchical text classification datasets [39] do not confirm this assumption. Rather, fine-tuning an SLM like the BERT-base model already outperforms it.

Regarding reasoning capabilities in LLMs, Guo, Zhang, and Qin [215] conclude that reasoning generally does not improve classification performance. Simpler strategies like Chain of Thought (CoT) yield consistent but limited gains of 1 to 3 percentage points on large models. More complex reasoning approaches often fail to outperform simpler baselines, and adding reasoning to small models may even degrade performance [215]. This comes at additional costs of using reasoning strategies, which consume up to two orders of magnitude more tokens for only marginal performance improvements. It seems that while sophisticated prompting strategies can be to some extent beneficial, longer reasoning does not necessarily lead to better classification performance and may even degrade performance.

C. Synthetic Text-graphs Hardly Bring an Advantage

Methods that generate a synthetic text graph and apply graph-based neural networks (GNNs) for text classification incur high training costs. First, the graph has to be computed. Second, a GNN has to be trained. For standard GNN methods, the whole graph has to fit into the GPU memory, and mini-batching is non-trivial but possible with dedicated sampling techniques for GNNs [216]. Computing the word-document graphs requires $\mathcal{O}(N^2)$ space, where N is the number of documents plus the vocabulary size, which is a hurdle for large-scale applications.

Our experiments show that simple BoW-based models like WideMLP and SVM already outperform graph-based models like TextGCN, HeteGCN, and Induct-GCN in the inductive text classification setting. An exception is 20ng, where MH-GAT performs better. An explanation may be that transforming

a text corpus into a graph cannot add any new information according to the data processing inequality [217].

The seminal paper on graph convolution by Kipf & Welling [156] argued that graph neural networks are most effective when the edges provide additional information that cannot be modeled otherwise. Therefore, it is important to distinguish between text-induced graphs for text classification, which seem to provide little to no gain, and tasks where the *natural* structure of the graph data provides more information than the mere text, e. g., citation networks. When extra information is encoded in the graph, graph neural networks are the state of the art [156], [165] and superior to MLPs that use only the node features and not the graph structure [218].

Our survey suggests that a graph induced from pure text generally does *not* improve text classification. Recently, Bugueño and de Melo [62] compared different document representations (Word2vec, GloVe, and frozen BERT) for graph neural networks. Using different datasets, they confirm that on most datasets, graph neural networks did *not* outperform a fine-tuned BERT regardless of the choice of input representation. The finding was also confirmed in a study on short text classification [219], where several graph-based methods have been compared to SLMs on six benchmark datasets (including R8 and MR) and four new datasets.

D. Using a Graph Encoder for the Hierarchy Hardly Brings an Advantage for Hierarchical Text Classification

In hierarchical text classification, we make similar observations as with single-label classification. Encoder-only models like DeBERTa and RoBERTa, the HBGL method (which incorporates the hierarchy into a standard BERT model), RADAr (which uses an autoregressive decoder instead of a classifier head), and HYDRA (which represents the hierarchy in level-wise losses), are overall best-performing depending on the dataset and metric. HiAGM uses a GNN to encode the class hierarchy but fails to outperform the hierarchy-agnostic sequence-based DeBERTa model. More modern models like HALB and HILL outperform DeBERTa-base, but fall behind other encoder-only models like HYDRA.

Ablations by Wang et al. [18] have shown that removing the graph encoder reduces the performance only by about 1 point (Micro/Macro-F1) [18]. Similarly, RADAr [40] and HYDRA [39] also provide empirical results that an explicit graph encoder is not needed for hierarchical text classification.

E. BERT Baselines are Often Undertuned

We found that BERT baselines are often undertuned in the literature. This can be declared as "baseline nerfing" and may be accidental [49]. We hope that our comparison sheds new light on the various proposed methods with solid BERT baselines. What we argue to be particularly problematic is omitting encoder-only baselines as soon as some prior work has a marginal advantage over BERT, as this practice prohibits readers from properly contextualizing the results, e. g., when the new method is also only marginally better than BERT. Based on the results of our quantitative comparison, we argue that simple baselines such as BERT, an MLP, or an SVM should not be omitted in text classification.

IX. Limitations

A limitation of our comparison may be the selection of datasets on which we base this comparative survey. Although the selection of specific datasets could potentially bias the comparison, we chose the most common datasets for maximum coverage of the methods. The current dataset collection was made to ensure broad coverage of approaches for single-label text classification, multi-label text classification, and hierarchical text classification. But we observe that the choice of multi-label datasets in the literature is more scattered than in the single-label case, which harms comparability. In order to balance this, we have included the most prominent multi-label datasets that provide a hierarchy, namely, WOS, NYT, and RCV1-V2. We also include datasets that go beyond the news domain, such as EconBiz, DBPedia, and even non-topical classification tasks, such as GoEmotions. For comparability, we have reported three variants of the F measure, namely sample-based, micro, and macro F1. Generally, we expect similar observations to be made on other text classification datasets because we already covered a wide range of text classification settings from long, medium, to short texts, topic and sentiment classification, and from single-label, multi-label, to hierarchical classification in the domains of forum postings, news, movie reviews, scholarly articles, and product reviews.

We further emphasize that the experimental datasets are limited to English. While word order is important in the English language, it is notable that methods that discard word order still work very well for topical text classification. We assume that BoW-based models perform even better for languages with a richer morphology, where word order is less important. It would be interesting to see to what extent our results of comparing BoW-based vs. sequence-based vs. graph-based vs. hierarchy-based methods for text classification transfer to other languages. Towards this direction, González-Carvajal and Garrido-Merchán [220] show that BERT outperforms classic TF-IDF BoW approaches on English, Chinese, and Portuguese text classification datasets. But other methods are yet to be considered. Another direction is to consider specifically designed text classifiers for a single language, e. g., in Chinese [221] where methods are tailored to the characteristics of Chinese characters, words, and radical information. Furthermore, there is room for an even larger coverage of datasets in future work [222].

Lastly, a major problem with evaluating generative language models is that benchmark datasets may have leaked into their training data [223]. It is not (always) clear if certain test sets are included in the language models' pre-training data.

X. Future Directions

As argued in the previous section, SLMs benefit from fine-tuning on a training set, while generative language models can do in-context learning with few examples. However, a large language model like Llama can also be fine-tuned for text classification [34]. This strengthens the view that the main distinction is whether the model is fine-tuned or not and hints at fine-tuning of generative language models as a direction of future work, e. g., via group relative policy optimization [224].

A different direction is to increase the model size of masked language models. That is, train larger versions of BERT on modern dataset collections. However, masked language models have lower "throughput" than causal language models. This is because causal language models obtain a training signal from every token, whereas masked language models only receive a training signal from masked tokens. Still, encoding left and right context has been shown to be important for text classification performance [34]. It remains unclear which strategy is better: training a large masked language model or unmasking a large causal language model during fine-tuning.

Existing studies on using LLMs are mostly considering single-label datasets, e. g., [25], [28], [41]. Expanding large language models for multi-label and hierarchical text classification raises interesting questions: how to feed the multi-label hierarchy into the input context of a language model, e. g., as in [225]. Existing approaches so far underperform, such as the 16-shot Retrieval-style ICL [36] or TaxMorph [213], of which the latter even modifies the provided taxonomy for the LLM. Thus, an open question is how to effectively use LLMs for multi-label classification and provide the hierarchy as input.

Another challenging setting is few-shot classification as in prompt-based large language models [24]. Here, generative LLMs can exploit their advantage by accessing large numbers of parameters trained on large amounts of data. As a study on single-label datasets showed [28], sufficient training data is needed for SLMs to outperform LLMs. Thus, an interesting direction may be to investigate approaches that allow SLMs to perform better in a few-shot regime, e. g., end-task-aware pre-training vs. fine-tuning after pre-training [226].

XI. Conclusion

The introduction of sequence-based transformer models, starting with the seminal BERT paper by Devlin et al. [20] in late 2018, has, as with other NLP tasks, strongly advanced the field. From our survey, we see that among the single-label, multi-label, and hierarchical text classification tasks, the top performers are fine-tuned language models. We argue that discriminative SLMs and generative LLMs have individual strengths, influenced by factors such as the availability of sufficient training data.

Reflecting back on our three central questions from Section I-B, we see that beyond the introduction of transformer models, today's methods operating on graphs generally provide no benefit over pre-trained language models. This is regardless of whether the graph is synthetically induced from text or if a hierarchy is provided with a multi-label dataset. Even worse, many new approaches fail to outperform straightforward baselines, such as an SVM or a multilayer perceptron.

Moreover, despite the astounding performance of LLMs in few-shot prompting, the best or close-to-best performance in single-label classification can be achieved through fine-tuning. This requires that sufficient training data is available [28]. The amount of training data needed depends on the task. It shows that despite advanced techniques for LLMs, like prompting and selecting in-context examples, text classification with fine-tuning SLMs is competitive. We argue that research in text classification should employ a pre-trained, encoder-only SLM such as a fine-tuned BERT as baseline.

Regarding the question of comparability, we found that, particularly for the strong baselines BERT and RoBERTa, the reported numbers vary across the literature. This is despite using benchmark datasets and apparently identical experimental settings. While it is not clear in all cases what caused these differences, our experiments have shown that already small changes in the learning rate can have a comparatively large effect on downstream performance. Thus, one has to carefully study a new method and check if experimental setups are comparable in order to derive conclusions about the advancements claimed.

Our study immediately impacts practitioners seeking to employ robust text classification models in research projects and industrial operational environments. Our recommendation to practitioners is to use a pre-trained SLM when sufficient training data are available. If this condition is not met, a frozen LLM with zero-shot or in-context classification may be used via few-shot learning. The trade-off lies between computational cost for fine-tuning vs. continued token cost for in-context learning.

Acknowledgments: We thank Yousef Younes and Gregor Donabauer for discussions and experiments on hierarchical and graph-based models. This work was co-funded by the DFG as part of the SmartER project (No. 515537520). L.G.P. acknowledges support from the Novo Nordisk Foundation under grant reference number NNF25OC0103204.

References

[1] F. Sebastiani, "Machine learning in automated text categorization," *ACM Comput. Surv.*, vol. 34, no. 1, 2002.
[2] A. Moreo, A. Esuli, and F. Sebastiani, "Learning to weight for text classification," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 2, 2020.
[3] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, "A survey of knowledge enhanced pre-trained language models," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 4, 2024.
[4] J. Fields, K. Chovanec, and P. Madiraju, "A Survey of Text Classification with Transformers: How wide? How large? How long? How accurate? How expensive? How safe?" *IEEE Access*, 2024.
[5] A. Zangari, M. Marcuzzo, M. Rizzo, L. Giudice, A. Albarelli, and A. Gasparetto, "Hierarchical text classification and its foundations: A review of current research," *Electronics*, vol. 13, no. 7, 2024.
[6] Q. Li, H. Peng, J. Li, C. Xia, R. Yang, L. Sun, P. S. Yu, and L. He, "A survey on text classification: From traditional to deep learning," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 2, 2022.
[7] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning-based text classification: A comprehensive review," *ACM Comput. Surv.*, vol. 54, no. 3, 2022.
[8] M. Bayer, M. Kaufhold, and C. Reuter, "A survey on data augmentation for text classification," *ACM Comput. Surv.*, vol. 55, no. 7, 2023.
[9] X. Zhou, R. Gururajan, Y. Li, R. Venkataraman, X. Tao, G. Bargshady, P. D. Barua, and S. Kondalsamy-Chennakesavan, "A survey on text classification and its applications," *Web Intell.*, vol. 18, no. 3, 2020.
[10] K. Kowsari, K. J. Meimandi, M. Heidarysafa, S. Mendu, L. E. Barnes, and D. E. Brown, "Text classification algorithms: A survey," *Inf.*, vol. 10, no. 4, 2019.
[11] A. I. Kadhim, "Survey on supervised machine learning techniques for automatic text classification," *Artif. Intell. Rev.*, vol. 52, no. 1, 2019.
[12] L. Wu, Y. Chen, K. Shen, X. Guo, H. Gao, S. Li, J. Pei, and B. Long, "Graph neural networks for natural language processing: a survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 2, 2023.
[13] M. Reusens, A. Stevens, J. Tonglet, J. D. Smedt, W. Verbeke, S. vanden Broucke, and B. Baesens, "Evaluating text classification: A benchmark study," *Expert Syst. Appl.*, vol. 254, 2024.
[14] W. L. Hamilton, *Graph Representation Learning*. Morgan and Claypool, 2020.

[15] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *AAAI*, 2019.

[16] Y. Zhang, X. Yu, Z. Cui, S. Wu, Z. Wen, and L. Wang, "Every document owns its structure: Inductive text classification via graph neural networks," in *ACL*, 2020.

[17] T. Jiang, D. Wang, L. Sun, Z. Chen, F. Zhuang, and Q. Yang, "Exploiting global and local hierarchies for hierarchical text classification," in *EMNLP*, 2022.

[18] Z. Wang, P. Wang, L. Huang, X. Sun, and H. Wang, "Incorporating hierarchy into text encoder: a contrastive learning approach for hierarchical text classification," in *ACL*, 2022.

[19] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, and X. B. Hu, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *ACM Trans. Knowl. Discov. Data*, vol. 18, 2024.

[20] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019.

[21] L. Galke and A. Scherp, "Bag-of-words vs. graph vs. sequence in text classification: Questioning the necessity of text-graphs and the surprising strength of a wide MLP," in *ACL*, 2022.

[22] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *JMLR*, vol. 21, 2020.

[23] OpenAI, "GPT-4 technical report," *arXiv:2303.08774*, 2023.

[24] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[25] X. Sun, X. Li, J. Li, F. Wu, S. Guo, T. Zhang, and G. Wang, "Text classification via large language models," in *EMNLP Findings*, 2023.

[26] A. Patel, B. Li, M. S. Rasooli, N. Constant, C. Raffel, and C. Callison-Burch, "Bidirectional language models are also few-shot learners," in *ICLR*. OpenReview.net, 2023.

[27] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *ICLR*, 2022.

[28] B. Pecher, I. Srba, and M. Bieliková, "Comparing specialised small and general large language models on text classification: 100 labelled samples to achieve break-even performance," in *EMNLP*. ACL, 2025.

[29] M. R. Qorib, G. Moon, and H. T. Ng, "Are decoder-only language models better than encoder-only language models in understanding word meaning?" in *ACL Findings*, 2024.

[30] M. J. J. Bucher and M. Martini, "Fine-tuned 'small' LLMs (still) significantly outperform zero-shot generative ai models in text classification," *arXiv:2406.08660*, 2024.

[31] P. Lepagnol, T. Gerald, S. Ghannay, C. Servan, and S. Rosset, "Small language models are good too: An empirical study of zero-shot classification," in *LREC/COLING*. ELRA and ICCL, 2024.

[32] C. ChunLiu, H. Zhang, K. Zhao, X. Ju, and L. Yang, "Llmembed: Rethinking lightweight llm's genuine function in text classification," in *ACL*, 2024.

[33] A. Edwards and J. Camacho-Collados, "Language models for text classification: Is in-context learning enough?" in *LREC-COLING*, 2024.

[34] Z. Li, X. Li, Y. Liu, H. Xie, J. Li, F. lee Wang, Q. Li, and X. Zhong, "Label supervised llama finetuning," *arXiv:2310.01208*, 2023.

[35] B. Hu, Q. Sheng, J. Cao, Y. Shi, Y. Li, D. Wang, and P. Qi, "Bad actor, good advisor: Exploring the role of large language models in fake news detection," in *AAAI*, 2024.

[36] H. Chen, Y. Zhao, Z. Chen, M. Wang, L. Li, M. Zhang, and M. Zhang, "Retrieval-style in-context learning for few-shot hierarchical text classification," *Trans. Assoc. Comput. Linguistics*, vol. 12, 2024.

[37] B. Bergner, A. Skliar, A. Royer, T. Blankevoort, Y. M. Asano, and B. E. Bejnordi, "Think big, generate quick: LLM-to-SLM for fast autoregressive decoding," *arXiv:2402.16844*, 2024.

[38] J. Zhou, C. Ma, D. Long, G. Xu, N. Ding, H. Zhang, P. Xie, and G. Liu, "Hierarchy-aware global model for hierarchical text classification," in *ACL*, 2020.

[39] F. Karl and A. Scherp, "HYDRA: A multi-head encoder-only architecture for hierarchical text classification," in *EMNLP*. ACL, 2025.

[40] Y. Younes, L. Galke, and A. Scherp, "RADAr: A transformer-based autoregressive decoder architecture for hierarchical text classification," in *ECAI*, 2024.

[41] M. Wang, Y. Zhang, C. Ren, Q. Li, P. Tiwari, B. Wang, and J. Qin, "Adaptive boosting LLMs for text classification," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–12, 2026.

[42] L. Yuan, Y. Chen, G. Cui, H. Gao, F. Zou, X. Cheng, H. Ji, Z. Liu, and M. Sun, "Revisiting out-of-distribution robustness in NLP: Benchmarks, analysis, and LLMs evaluations," in *NeurIPS*, 2023.

[43] X. Li, X. Zhu, Z. Ma, X. Liu, and S. Shah, "Are ChatGPT and GPT-4 general-purpose solvers for financial text analytics? an examination on several typical tasks," *arXiv:2305.05862*, 2023.

[44] H. Yu, Z. Yang, K. Pelrine, J. F. Godbout, and R. Rabbany, "Open, closed, or small language models for text classification?" *arXiv:2308.10092*, 2023.

[45] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*. Curran Associates, Inc., 2022.

[46] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang, "P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks," in *ACL (Volume 2: Short Papers)*, 2022.

[47] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *EMNLP*, 2021.

[48] M. F. Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *RecSys*. ACM, 2019.

[49] G. Leech, J. J. Vazquez, M. Yagudin, N. Kupper, and L. Aitchison, "Questionable practices in machine learning," *arXiv:2407.12220*, 2024.

[50] R. Nallapati, "Discriminative models for information retrieval," in *SIGIR*. ACM, 2004.

[51] X. Liu, X. You, X. Zhang, J. Wu, and P. Lv, "Tensor graph convolutional networks for text classification," in *AAAI/IAAI/EAAI*, 2020.

[52] R. Ragesh, S. Sellamanickam, A. Iyer, R. Bairi, and V. Lingam, "HeteGCN: Heterogeneous graph convolutional networks for text classification," in *WSDM*. ACM, 2021.

[53] F. Shehzad and D. Jannach, "Everyone's a winner! on hyperparameter tuning of recommendation models," in *RecSys*. ACM, 2023.

[54] L. Galke, A. Diera, B. X. Lin, B. Khera, T. Meuser, T. Singhal, F. Karl, and A. Scherp, "Are we really making much progress? bag-of-words vs. sequence vs. graph vs. hierarchy for single- and multi-label text classification," *arXiv:2204.03954*, 2023.

[55] D. Shen, G. Wang, W. Wang, M. R. Min, Q. Su, Y. Zhang, C. Li, R. Henao, and L. Carin, "Baseline needs more love: On simple word-embedding-based models and associated pooling mechanisms," in *ACL*, 2018.

[56] R. You, Z. Zhang, Z. Wang, S. Dai, H. Mamitsuka, and S. Zhu, "AttentionXML: Label Tree-based Attention-Aware Deep Model for High-Performance Extreme Multi-Label Text Classification," in *NeurIPS*, vol. 32. Curran Associates, Inc., 2019.

[57] J. Zhang, W.-C. Chang, H.-F. Yu, and I. Dhillon, "Fast Multi-Resolution Transformer Fine-tuning for Extreme Multi-label Text Classification," in *NeurIPS*, vol. 34. Curran Associates, Inc., 2021.

[58] S. Kharbanda, A. Banerjee, D. Gupta, A. Palrecha, and R. Babbar, "InceptionXML: A Lightweight Framework with Synchronized Negative Sampling for Short Text Extreme Classification," in *SIGIR*, 2023.

[59] S. Kharbanda, A. Banerjee, E. Schultheis, and R. Babbar, "CascadeXML: Rethinking Transformers for End-to-end Multi-resolution Training in Extreme Multi-label Classification," in *NeurIPS*, 2022.

[60] K. Wang, Y. Ding, and S. C. Han, "Graph neural networks for text classification: a survey," *Artif. Intell. Rev.*, vol. 57, no. 8, 2024.

[61] J. M. Duarte and L. Berton, "A review of semi-supervised learning for text classification," *Artif. Intel.l. Rev.*, 2023.

[62] M. Bugueño and G. de Melo, "Connecting the Dots: What Graph-Based Text Representations Work Best for Text Classification using Graph Neural Networks?" *arXiv:2305.14578*, 2023.

[63] A. N. Tarekegn, M. Giacobini, and K. Michalak, "A review of methods for imbalanced multi-label classification," *Pattern Recognit.*, 2021.

[64] D. Raihan, "Deep learning techniques for text classification," 2021.

[65] M. Qaraei, S. Khandagale, and R. Babbar, "Why state-of-the-art deep learning barely works as good as a linear classifier in extreme multi-label text classification," in *ESANN*, 2020.

[66] W. Liu, H. Wang, X. Shen, and I. W. Tsang, "The emerging trends of multi-label learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2022.

[67] F. Mai, L. Galke, and A. Scherp, "Using deep learning for title-based semantic subject indexing to reach competitive performance to full-text," in *JCDL*. ACM, 2018.

[68] L. Galke, F. Mai, A. Schelten, D. Brunsch, and A. Scherp, "Using titles vs. full-text as source for automated semantic document annotation," in *Knowledge Capture Conf., K-CAP 2017*. ACM, 2017.

[69] D. Zhang, J. Wang, E. Yilmaz, X. Wang, and Y. Zhou, "Bayesian performance comparison of text classifiers," in *SIGIR*. ACM, 2016.

[70] R. Sharma, S. Kumar, and A. Kumar, "Read: Reinforcement-based adversarial learning for text classification with limited labeled data," *arXiv:2501.08035*, 2025.

[71] C. Petridis, "Text classification: Neural networks vs machine learning models vs pre-trained models," *arXiv:2412.21022*, 2024.

[72] Q. Peng, X. Luo, Y. Yuan, F. Gu, H. Shen, and Z. Huang, "A text classification method combining in-domain pre-training and prompt learning for the steel e-commerce industry," *Int. Journal of Web Information Systems*, vol. 21, no. 1, 2025.

[73] X. Jia, M. Jiang, Y. Dong, F. Zhu, H. Lin, Y. Xin, and H. Chen, "Multimodal heterogeneous graph attention network," *Neural Comput. Appl.*, vol. 35, no. 4, 2023.

[74] N. Niraula, S. Ayhan, B. Chidambaram, and D. Whyatt, "Multi-label classification with generative large language models," in *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conf. (DASC)*, 2024.

[75] G. Sun, J. Li, Y. Cheng, and Z. Zhang, "Lmtcsg: Multilabel text classification combining sequence-based and gnn-based features," *IEEE Trans. on Industrial Informatics*, 2024.

[76] V. K. Agbesi, W. Chen, S. B. Yussif, C. C. Ukwuoma, Y. H. Gu, and M. A. Al-antari, "Mutcelm: An optimal multi-textcnn-based ensemble learning for text classification," *Heliyon*, vol. 10, no. 19, 2024.

[77] X. Xu, M. Hu, Y. Wang, W. Luo, S. Liu, Z. Luo, and Y. Tan, "Pliclass: Weakly supervised text classification with iterative training and denoisy inference," in *ICANN 2024*. Springer, 2024.

[78] L. Gao, "Harnessing the intrinsic knowledge of pretrained language models for challenging text classification settings," *arXiv:2408.15650*, 2024.

[79] J. Wei and X. Sun, "Study on text classification model combining bert and convolutional neural network," *Mathematical Modeling and Algorithm Application*, vol. 2, no. 3, 2024.

[80] L. Yu, H. Li, K. Chen, and L. Shou, "Boka: Bayesian optimization based knowledge amalgamation for multi-unknown-domain text classification," in *SIGKDD*. ACM, 2024.

[81] L. Dai, Y. Yin, E. Chen, and H. Xiong, "Unifying graph retrieval and prompt tuning for graph-grounded text classification," in *SIGIR*, 2024.

[82] S. Jamshidi, M. Mohammadi, S. Bagheri, H. E. Najafabadi, A. Rezvanian, M. Gheisari, M. Ghaderzadeh, A. S. Shahabi, and Z. Wu, "Effective text classification using BERT, MTM LSTM, and DT," *Data and Knowl. Eng.*, vol. 151, 2024.

[83] T. Thaminkaew, P. Lertvittayakumjorn, and P. Vateekul, "Prompt-based label-aware framework for few-shot multi-label text classification," *IEEE Access*, vol. 12, 2024.

[84] K. Ly, Y. Kashnitsky, S. Chamezopoulos, and V. V. Krzhizhanovskaya, "Article classification with graph neural networks and multigraphs," in *LREC/COLING*, 2024.

[85] L. Zhou, W. Chen, Y. Cao, D. Zeng, W. Liu, and H. Qu, "MLPs compass: What is learned when MLPs are combined with PLMs?" *arXiv:2401.01667*, 2024.

[86] T. Thaminkaew, P. Lertvittayakumjorn, and P. Vateekul, "Prompt-based label-aware framework for few-shot multi-label text classification," *IEEE Access*, vol. 12, 2024.

[87] Y. Zhu, Y. Wang, J. Qiang, and X. Wu, "Prompt-learning for short text classification," *IEEE Trans. on Knowl. and Data Eng.*, 2023.

[88] Y. Chae and T. Davidson, "Large language models for text classification: From zero-shot learning to fine-tuning," *Open Science Foundation*, vol. 10, 2023.

[89] Y. Zhang, H. Cheng, Z. Shen, X. Liu, Y. Wang, and J. Gao, "Pre-training multi-task contrastive learning models for scientific literature understanding," in *EMNLP Findings*, 2023.

[90] H. Yang, N. Wang, L. Yang, W. Liu, and S. Wang, "Research on the automatic subject-indexing method of academic papers based on climate change domain ontology," *Sustainability*, vol. 15, no. 5, 2023.

[91] C. Tan, Y. Ren, and C. Wang, "An adaptive convolution with label embedding for text classification," *Appl. Intell.*, vol. 53, no. 1, 2023.

[92] D. Zong and S. Sun, "BGNN-XML: Bilateral graph neural networks for extreme multi-label text classification," *IEEE Trans. on Knowl. and Data Eng.*, 2022.

[93] Q. Tran, K. Shpileuskaya, E. Zaunseder, L. Putzar, and S. Blankenburg, "Comparing the robustness of classical and deep learning techniques for text classification," in *IJCNN*. IEEE, 2022.

[94] T. Liu, Y. Hu, B. Wang, Y. Sun, J. Gao, and B. Yin, "Hierarchical graph convolutional networks for structured long document classification," *IEEE Trans. on Neural Networks and Learning Systems*, 2022.

[95] C. Breazzano, D. Croce, and R. Basili, "Multi-task and generative adversarial learning for robust and sustainable text classification," in *AIxIA 2021*. Springer, 2021.

[96] Z. Yu, D. Jin, J. Wei, Y. Li, Z. Liu, Y. Shang, J. Han, and L. Wu, "Teko: Text-rich graph neural networks with external knowledge," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 35, no. 10, 2024.

[97] S. Ghosh, S. Maji, and M. S. Desarkar, "Supervised graph contrastive pretraining for text classification," *arXiv:2112.11389*, 2021.

[98] I. Chalkidis, M. Fergadiotis, S. Kotitsas, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "An empirical study on large-scale multi-label text classification including few and zero-shot labels," in *EMNLP*, 2020.

[99] A. Conneau, H. Schwenk, L. Barrault, and Y. LeCun, "Very deep convolutional networks for text classification," in *EACL*, 2017.

[100] S. Gururangan, T. Dang, D. Card, and N. A. Smith, "Variational pretraining for semi-supervised text classification," in *ACL*, 2019.

[101] P. Zhou, Z. Qi, S. Zheng, J. Xu, H. Bao, and B. Xu, "Text classification improved by integrating bidirectional LSTM with two-dimensional max pooling," in *COLING*, 2016.

[102] S. Lai, L. Xu, K. Liu, and J. Zhao, "Recurrent convolutional neural networks for text classification," in *AAAI*, 2015.

[103] B. Parlak, "A novel feature and class-based globalization technique for text classification," *Multimedia Tools and Applications*, 2023.

[104] G. Donabauer and U. Kruschwitz, "Token-level graphs for short text classification," *arXiv:2412.12754*, 2024.

[105] N. Stylianou, D. Chatzakou, T. Tsikrika, S. Vrochidis, and I. Kompatsiaris, "Domain-aligned data augmentation for low-resource and imbalanced text classification," in *ECIR*. Springer, 2023.

[106] Y. Wahba, N. H. Madhavji, and J. Steinbacher, "A comparison of SVM against pre-trained language models (PLMs) for text classification tasks," in *LOD (2)*. Springer, 2022.

[107] Z. Sun, A. Harit, A. I. Cristea, J. Yu, L. Shi, and N. A. Moubayed, "Contrastive learning with heterogeneous graph attention networks on short text classification," in *IJCNN*. IEEE, 2022.

[108] A. Pal, M. Selvakumar, and M. Sankarasubbu, "MAGNET: multi-label text classification using attention-based graph neural network," in *ICAART*. SCITEPRESS, 2020.

[109] A. Moreo, A. Esuli, and F. Sebastiani, "Learning to weight for text classification," *IEEE Trans. on Knowl. and Data Eng.*, vol. 32, 2020.

[110] S. Canuto, D. X. Sousa, M. A. Gonçalves, and T. C. Rosa, "A Thorough Evaluation of Distance-Based Meta-Features for Automated Text Classification," *IEEE Trans. Knowl. Data Eng.*, vol. 30, 2018.

[111] W. Huang, C. Liu, B. Xiao, Y. Zhao, Z. Pan, Z. Zhang, X. Yang, and G. Liu, "Exploring label hierarchy in a generative way for hierarchical text classification," in *COLING*. Computational Linguistics, 2022.

[112] R. Sarasu, K. K. Thyagharajan, and N. R. Shanker, "SF-CNN: Deep text classification and retrieval for text documents," *Intelligent Automation & Soft Computing*, vol. 35, no. 2, 2023.

[113] P. Yuan, Y. Chen, H. Jin, and L. Huang, "MSVM-kNN: Combining SVM and k-NN for multi-class text classification," in *IEEE Int. Workshop on Semantic Computing and Systems*, 2008.

[114] W. Liang, T. Zhang, H. Liu, and F. Zhang, "SELP: A semantically-driven approach for separated and accurate class prototypes in few-shot text classification," in *ACL Findings*, 2024.

[115] H. Liu, S. Zhao, X. Zhang, F. Zhang, W. Wang, F. Ma, H. Chen, H. Yu, and X. Zhang, "Liberating seen classes: Boosting few-shot and zero-shot text classification via anchor generation and classification reframing," in *AAAI*, 2024.

[116] Z. Xie, Y. Lv, Y. Song, and Q. Wang, "Data labeling through the centralities of co-reference networks improves the classification accuracy of scientific papers," *Journal of Informetrics*, 2024.

[117] K. Zheng, Y. Wang, Q. Yao, and D. Dou, "Simplified graph learning for inductive short text classification," in *EMNLP*, 2022.

[118] Y. Wang, S. Wang, Q. Yao, and D. Dou, "Hierarchical heterogeneous graph representation learning for short text classification," in *EMNLP*, 2021.

[119] Z. Tan, J. Chen, Q. Kang, M. Zhou, A. Abusorrah, and K. Sedraoui, "Dynamic Embedding Projection-Gated Convolutional Neural Networks for Text Classification," *IEEE Trans. on Neural Networks and Learning Systems*, vol. 33, no. 3, 2022.

[120] Y. Huang, B. Giledereli, A. Köksal, A. Özgür, and E. Ozkirimli, "Balancing methods for multi-label text classification with long-tailed class distribution," in *EMNLP*, 2021.

[121] J. Attieh and J. Tekli, "Supervised term-category feature weighting for improved text classification," *Knowl.-Based Systems*, vol. 261, 2023.

[122] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *NeurIPS*, 2013.

[123] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *EMNLP*, 2014.

[124] M. Iyyer, V. Manjunatha, J. L. Boyd-Graber, and H. D. III, "Deep unordered composition rivals syntactic methods for text classification," in *ACL*, 2015.

[125] N. Kalchbrenner, E. Grefenstette, and P. Blunsom, "A convolutional neural network for modelling sentences," in *ACL*, 2014.

[126] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, 2017.

[127] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *EACL*, 2017.

[128] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, 1997.

[129] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv:1409.0473*, 2014.

[130] P. Liu, X. Qiu, and X. Huang, "Recurrent neural network for text classification with multi-task learning," *arXiv:1605.05101*, 2016.

[131] P. Yang, X. Sun, W. Li, S. Ma, W. Wu, and H. Wang, "SGM: sequence generation model for multi-label classification," in *COLING*, 2018.

[132] C. Yu, Y. Shen, and Y. Mao, "Constrained sequence-to-tree generation for hierarchical text classification," in *SIGIR*, 2022.

[133] Y. Huang, Y. Chen, and Y. Chen, "Contexting: Granting document-wise contextual embeddings to graph neural networks for inductive text classification," in *COLING*, 2022.

[134] K. Zhao, L. Huang, R. Song, Q. Shen, and H. Xu, "A sequential graph neural network for short text classification," *Algorithms*, vol. 14, 2021.

[135] S. Lyu and J. Liu, "Combine convolution with recurrent networks for text classification," *arXiv:2006.15795*, 2020.

[136] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[137] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv:1907.11692*, 2019.

[138] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.

[139] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," in *ICLR*. OpenReview.net, 2020.

[140] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: decoding-enhanced bert with disentangled attention," in *ICLR*, 2021.

[141] Y. Sun, S. Wang, Y. Li, S. Feng, H. Tian, H. Wu, and H. Wang, "ERNIE 2.0: A continual pre-training framework for language understanding," in *AAAI*. AAAI Press, 2020.

[142] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing," in *ICLR*. OpenReview.net, 2023.

[143] K. Clark, M. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: pre-training text encoders as discriminators rather than generators," in *ICLR*. OpenReview.net, 2020.

[144] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *ICLR*. OpenReview.net, 2019.

[145] Z. Yu, H. Li, and J. Feng, "Enhancing text classification with attention matrices based on BERT," *Expert Systems*, 2023.

[146] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas *et al.*, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," *arXiv:2412.13663*, 2024.

[147] J. Nam, E. Loza Mencía, H. J. Kim, and J. Fürnkranz, "Maximizing Subset Accuracy with Recurrent Neural Networks in Multi-label Classification," in *NeurIPS*. Curran Associates, Inc., 2017.

[148] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *ACL*, 2020.

[149] L. Team, "The Llama 3 herd of models," *CoRR*, vol. abs/2407.21783, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.21783

[150] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, 2019.

[151] B. Hou, J. O'Connor, J. Andreas, S. Chang, and Y. Zhang, "Prompt-Boosting: Black-box text classification with ten forward passes," in *ICML*, 2023.

[152] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava *et al.*, "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv:2307.09288*, 2023.

[153] Y. Ohsawa, N. E. Benson, and M. Yachida, "Keygraph: Automatic indexing by co-occurrence graph based on building construction metaphor," in *IEEE Forum on Res. and Techn. Adv. Digit. Libr.*, 1998.

[154] S. Rose, D. Engel, N. Cramer, and W. Cowley, "Automatic keyword extraction from individual documents," in *Text Mining. Applications and Theory*. Wiley, 2010.

[155] L. Zhang, J. Ding, Y. Xu, Y. Liu, and S. Zhou, "Weakly-supervised text classification based on keyword graph," in *EMNLP*, 2021.

[156] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*. OpenReview.net, 2017.

[157] K. Ding, J. Wang, J. Li, D. Li, and H. Liu, "Be more with less: Hypergraph attention networks for inductive text classification," in *EMNLP*, 2020.

[158] T. Yang, L. Hu, C. Shi, H. Ji, X. Li, and L. Nie, "HGAT: heterogeneous graph attention networks for semi-supervised short text classification," *ACM Trans. Inf. Syst.*, vol. 39, no. 3, 2021.

[159] Y. Liu, R. Guan, F. Giunchiglia, Y. Liang, and X. Feng, "Deep attention diffusion graph neural networks for text classification," in *EMNLP*, 2021.

[160] Z. Ye, G. Jiang, Y. Liu, Z. Li, and J. Yuan, "Document and word representations generated by graph convolutional network and BERT for short text classification," in *ECAI*. IOS Press, 2020.

[161] Z. Deng, C. Sun, G. Zhong, and Y. Mao, "Text Classification with Attention Gated Graph Neural Network," *Cogn. Comput.*, vol. 14, 2022.

[162] K. Wang, S. C. Han, and J. Poon, "InducT-GCN: Inductive graph convolutional networks for text classification," in *26th Int. Conf. on Pattern Recognition, ICPR*. IEEE, 2022.

[163] L. Huang, D. Ma, S. Li, X. Zhang, and H. Wang, "Text level graph neural network for text classification," in *EMNLP-IJCNLP*, 2019.

[164] J. Tang, M. Qu, and Q. Mei, "PTE: predictive text embedding through large-scale heterogeneous text networks," in *SIGKDD*. ACM, 2015.

[165] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.

[166] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *JMLR*, vol. 3, 2003.

[167] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated graph sequence neural networks," in *ICLR*, 2016.

[168] Y. Jin, W. Yin, H. Wang, and F. He, "Capturing word positions does help: A multi-element hypergraph gated attention network for document classification," *Expert Syst. Appl.*, vol. 251, 2024.

[169] Y. Lin, Y. Meng, X. Sun, Q. Han, K. Kuang, J. Li, and F. Wu, "BertGCN: Transductive text classification by combining GNN and BERT," in *ACL/IJCNLP Findings*, 2021.

[170] X. Xu, T. Zhang, C. Xu, and Z. Cui, "Circulant tensor graph convolutional network for text classification," in *Pattern Recognition - 6th Asian Conf., ACPR*. Springer, 2021.

[171] J. Shi, X. Wu, X. Liu, W. Lu, and S. Li, "Inductive Light Graph Convolution Network for Text Classification Based on Word-Label Graph," in *Intelligent Information Processing XI*. Springer, 2022.

[172] X. Li, X. Wu, Z. Luo, Z. Du, Z. Wang, and C. Gao, "Integration of global and local information for text classification," *Neural Computing and Applications*, 2022.

[173] X. Wang, C. Wang, H. Yang, X. Zhang, Q. Shen, K. Ji, Y. Wu, and H. Zhan, "KGAT: An enhanced graph-based model for text classification," in *Nat. Lang. Proc. Chinese Comput.* Springer, 2022.

[174] Y. Piao, S. Lee, D. Lee, and S. Kim, "Sparse structure learning via graph neural networks for inductive document classification," in *AAAI*, 2022.

[175] S. Yin, P. Zhu, X. Wu, J. Huang, X. Li, Z. Wang, and C. Gao, "Integrating information by Kullback–Leibler constraint for text classification," *Neural Computing and Applications*, 2023.

[176] J. Shen, W. Qiu, Y. Meng, J. Shang, X. Ren, and J. Han, "TaxoClass: Hierarchical multi-label text classification using only class names," in *NAACL*, 2021.

[177] H. Peng, J. Li, S. Wang, L. Wang, Q. Gong, R. Yang, B. Li, P. S. Yu, and L. He, "Hierarchical Taxonomy-Aware and Attentional Graph Capsule RCNNs for Large-Scale Multi-Label Text Classification," *IEEE Trans. on Knowl. and Data Eng.*, vol. 33, no. 6, 2021.

[178] H. Dong, V. Suárez-Paniagua, W. Whiteley, and H. Wu, "Explainable automated coding of clinical notes using hierarchical label-wise attention networks and label embedding initialisation," *J. Biomed. Informatics*, vol. 116, 2021.

[179] X. Zhang, J. Xu, C. Soh, and L. Chen, "LA-HCN: label-based attention for hierarchical multi-label text classification neural network," *Expert Syst. Appl.*, vol. 187, 2022.

[180] H. Liu, D. Zhang, B. Yin, and X. Zhu, "Improving pretrained models for zero-shot multi-label text classification through reinforced label hierarchy reasoning," in *NAACL-HLT*, 2021.

[181] L. Zhang, J. Ding, Y. Xu, Y. Liu, and S. Zhou, "Weakly-supervised text classification based on keyword graph," in *EMNLP*, 2021.

[182] H. Xiao, X. Liu, and Y. Song, "Efficient path prediction for semi-supervised and weakly supervised hierarchical text classification," in *WWW*. ACM, 2019.

[183] W. Yin, J. Hay, and D. Roth, "Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach," in *EMNLP/IJCNLP*, 2019.

[184] H. Chen, Q. Ma, Z. Lin, and J. Yan, "Hierarchy-aware label semantics matching network for hierarchical text classification," in *ACL/IJCNLP*, 2021.

[185] Y. Liu, K. Zhang, Z. Huang, K. Wang, Y. Zhang, Q. Liu, and E. Chen, "Enhancing hierarchical text classification through knowledge graph integration," in *ACL Findings*, 2023.

[186] J. Zhang, Y. Li, F. Shen, C. Xia, H. Tan, and Y. He, "Hierarchy-aware and label balanced model for hierarchical text classification," *Knowl. Based Syst.*, vol. 300, 2024.

[187] H. Zhu, J. Wu, R. Liu, Y. Hou, Z. Yuan, S. Li, Y. Pan, and K. Xu, "HILL: Hierarchy-aware information lossless contrastive learning for hierarchical text classification," in *NAACL*, 2024.

[188] C. Zhang, L. Dai, C. Liu, and L. Zhang, "HGBL: A fine granular hierarchical multi-label text classification model," *Neural Process. Lett.*, vol. 57, no. 1, 2025.

[189] K. Bache and M. Lichman, "UCI machine learning repository," 2013, University of California, Irvine, School of Information and Computer Sciences. [Online]. Available: http://archive.ics.uci.edu/ml

[190] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," in *ACL*, 2005.

[191] D. D. Lewis, Y. Yang, T. G. Rose, and F. Li, "RCV1: A new benchmark collection for text categorization research," *JMLR*, vol. 5, 2004.

[192] D. Demszky, D. Movshovitz-Attias, J. Ko, A. S. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A dataset of fine-grained emotions," in *ACL*, 2020.

[193] J. McAuley and J. Leskovec, "Hidden factors and hidden topics: Understanding rating dimensions with review text," in *RecSys*, 2013.

[194] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. van Kleef, S. Auer, and C. Bizer, "DBpedia - a large-scale, multilingual knowledge base extracted from Wikipedia," *Semantic Web*, vol. 6, 2015.

[195] E. Sandhaus, "The New York Times Annotated Corpus," 2008, linguistic Data Consortium, Philadelphia, 6(12):e26752.

[196] K. Kowsari, D. E. Brown, M. Heidarysafa, K. J. Meimandi, M. S. Gerber, and L. E. Barnes, "HDLTex: Hierarchical deep learning for text classification," in *ICMLA*. IEEE, 2017.

[197] M. Li, Y. Liu, F. Giunchiglia, X. Feng, and R. Guan, "Simple-sampling and hard-mixup with prototypes to rebalance contrastive learning for text classification," *arXiv:2405.11524*, 2024.

[198] Y. Zhang and B. C. Wallace, "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," in *IJCNLP*, 2017.

[199] Y. Kim, "Convolutional neural networks for sentence classification," in *ACL*, 2014.

[200] F. Wu, A. H. S. Jr., T. Zhang, C. Fifty, T. Yu, and K. Q. Weinberger, "Simplifying graph convolutional networks," in *ICML*, 2019.

[201] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *ICLR*, 2015.

[202] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 8, 2014.

[203] G. Tsoumakas and I. Katakis, "Multi-label classification: An overview," *Int. Journal of Data Warehousing and Mining*, vol. 3, 2009.

[204] H. Wu, Y. Zhang, Z. Han, Y. Hou, L. Wang, S. Liu, Q. Gong, and Y. Ge, "Quartet logic: A four-step reasoning (QLFR) framework for advancing short text classification," *arXiv:2401.03158*, 2024.

[205] Y. Mao, P. Liu, T. Cui, C. Liu, and D. You, "Low-resource fast text classification based on intra-class and inter-class distance calculation," in *COLING*, 2025.

[206] D. Chai, W. Wu, Q. Han, F. Wu, and J. Li, "Description based text classification with reinforcement learning," in *ICML*, 2020.

[207] V. Jain, M. Rungta, Y. Zhuang, Y. Yu, Z. Wang, M. Gao, J. Skolnick, and C. Zhang, "HiGen: Hierarchy-aware sequence generation for hierarchical text classification," in *EACL*. ACL, 2024.

[208] C. Yu, Y. Shen, and Y. Mao, "Constrained sequence-to-tree generation for hierarchical text classification," in *SIGIR*. ACM, 2022.

[209] F. Torba, C. Gravier, C. Laclau, A. Kammoun, and J. Subercaze, "Decoding the hierarchy: A hybrid approach to hierarchical multi-label text classification," in *ECIR*. Springer, 2025.

[210] F. Kong, R. Zhang, and Z. Wang, "LH-Mix: Local hierarchy correlation guided mixup over hierarchical prompt tuning," in *SIGKDD*, 2025.

[211] S. C. L. U, J. He, V. Gutiérrez-Basulto, and J. Z. Pan, "Instances and labels: Hierarchy-aware joint supervised contrastive learning for hierarchical multi-label text classification," in *EMNLP Findings*, 2023.

[212] A. Kumar and D. Toshniwal, "HyILR: Hyperbolic instance-specific local relationships for hierarchical text classification," in *Student Research Workshop at ACL*. ACL, 2025.

[213] J. Golde, N. P. Jedema, R. Krishnan, and P. Le, "Hierarchical text classification with LLM-refined taxonomies," in *EACL*. ACL, 2026.

[214] A. Yehudai and E. Bendel, "When LLMs are unfit use FastFit: Fast and effective text classification with many classes," *arXiv:2404.12365*, 2024.

[215] X. Guo, Y. Zhang, and J. Qin, "TextReasoningBench: Does reasoning really improve text classification in large language models?" 2026. [Online]. Available: https://arxiv.org/abs/2603.19558

[216] M. Fey, J. E. Lenssen, F. Weichert, and J. Leskovec, "GNNAutoScale: Scalable and expressive graph neural networks via historical embeddings," in *ICML*, 2021.

[217] T. M. Cover and J. A. Thomas, *Elements of Information Theory*. Wiley, 1991.

[218] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *arXiv:1811.05868*, 2018.

[219] F. Karl and A. Scherp, "Transformers are short-text classifiers," in *Int. Cross-Domain Conf., CD-MAKE*. Springer, 2023.

[220] S. González-Carvajal and E. C. Garrido-Merchán, "Comparing BERT against traditional machine learning text classification," *arXiv:2005.13012*, 2020.

[221] J. Guo, B. Zhao, H. Liu, Y. Liu, and Q. Zhong, "Supervised contrastive learning with term weighting for improving chinese text classification," *Tsinghua Science and Technology*, vol. 28, no. 1, 2023.

[222] K. Bhatia, K. Dahiya, H. Jain, P. Kar, A. Mittal, Y. Prabhu, and M. Varma, "The extreme classification repository: Multi-label datasets and code," 2016. [Online]. Available: http://manikvarma.org/downloads/XC/XMLRepository.html

[223] S. Balloccu, P. Schmidtová, M. Lango, and O. Dusek, "Leak, Cheat, Repeat: Data Contamination and Evaluation Malpractices in Closed-Source LLMs," in *EACL*, 2024.

[224] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv:2402.03300*, 2024.

[225] B. Fatemi, J. Halcrow, and B. Perozzi, "Talk like a graph: Encoding graphs for large language models," in *ICLR*, 2024.

[226] L. M. Dery, P. Michel, A. Talwalkar, and G. Neubig, "Should we be pre-training? An argument for end-task aware training as an alternative," in *ICLR*, 2022.

Ansgar Scherp is Full Professor for Data Science and Big Data Analytics with Ulm University, Germany, since 2020. He has long-standing experience in interdisciplinary projects in Germany as well as the USA and UK. His main interests are in graph machine learning and natural language processing, often with applications in information retrieval and extraction.

Andor Diera, Fabian Karl, Bao Xin Lin, Bhakti Khera, Tim Meuser, and Tushar Singhal contributed as students in computer science and cognitive systems at Ulm University. Bao passed away during the preparation of this manuscript; her colleagues and co-authors dedicate this work to her memory.

Lukas Galke Poech (formerly published as L. Galke) received the Ph.D. degree in computer science from Kiel University, Germany in 2023. He was a Doctoral Researcher with Kiel University, Germany and with the Leibniz Information Centre for Economics, and from 2022 to 2024 a Postdoctoral Researcher at the Max Planck Institute for Psycholinguistics, Nijmegen. Since 2024 he has been with the University of Southern Denmark where he is currently an Associate Professor. His research interests include AI safety, interpretability, and representation learning in language models. For more information, see https://lgalke.github.io.